Програмування
Лекція 1. Вступ. Загальні поняття програмування. Перша програма мовою C++.

02.09.2025

Дуже важливо!

Вміння програмувати НЕ є передумовою успішного опанування курсу.

Передумовами успішного опанування є системність та наполегливість у навчанні та бажання вчитись.

У кожній новій справі спочатку буває важко, але з часом приходять і результати.

Якщо виникають питання, то розумні їх відразу задають. Менш розумні залишаються зі своїми питаннями наодинці.
Будьте розумними!

Вступ

Предмет курсу

Базові концепції мов програмування:

типи даних, оператори, змінні, інструкції,

організація пам'яті програми тощо

та як ними користуватись на прикладі мови C++ (переважно C++20, можливо з деякими нововведеннями C++23, якщо вони підтримуватимуться основними лінійками компіляторів).

Вступ

Предмет курсу

Базові концепції мов програмування:

типи даних, оператори, змінні, інструкції,
організація пам'яті програми тощо

та як ними користуватись на прикладі мови C++ (переважно C++20, можливо з деякими нововведеннями C++23, якщо вони підтримуватимуться основними лінійками компіляторів).

Алгоритмів існує дуже багато, вони достатньо спеціалізовані і не є предметом розгляду вступного курсу програмування. Але деякі все-таки розглянемо.

Вступ

Предмет курсу

Базові концепції мов програмування:

типи даних, оператори, змінні, інструкції,
організація пам'яті програми тощо

та як ними користуватись на прикладі мови C++ (переважно C++20, можливо з деякими нововведеннями C++23, якщо вони підтримуватимуться основними лінійками компіляторів).

Алгоритмів існує дуже багато, вони достатньо спеціалізовані і не є предметом розгляду вступного курсу програмування. Але деякі все-таки розглянемо.

Структури даних — способи організації даних у пам'яті.

Вступ

Предмет курсу

Базові концепції мов програмування:

типи даних, оператори, змінні, інструкції,
організація пам'яті програми тощо

та як ними користуватись на прикладі мови C++ (переважно C++20, можливо з деякими нововведеннями C++23, якщо вони підтримуватимуться основними лінійками компіляторів).

Алгоритмів існує дуже багато, вони достатньо спеціалізовані і не є предметом розгляду вступного курсу програмування. Але деякі все-таки розглянемо.

Структури даних — способи організації даних у пам'яті.

Стандартна бібліотека мови C++.

Вступ

Предмет курсу

Базові концепції мов програмування:

типи даних, оператори, змінні, інструкції,
організація пам'яті програми тощо

та як ними користуватись на прикладі мови C++ (переважно C++20, можливо з деякими нововведеннями C++23, якщо вони підтримуватимуться основними лінійками компіляторів).

Алгоритмів існує дуже багато, вони достатньо спеціалізовані і не є предметом розгляду вступного курсу програмування. Але деякі все-таки розглянемо.

Структури даних — способи організації даних у пам'яті.

Стандартна бібліотека мови C++.

Технологія розроблення програмного забезпечення, зокрема система організації коду, є невід'ємною складовою сучасного програмування. Певні елементи розглядатись будуть.

Вступ

Перший семестр: складові

- контрольна робота № 1, max = 15/8 балів
- контрольна робота № 2, max = 15/8 балів
- програмний проєкт № 1, max = 10/5 балів
- програмний проєкт № 2, max = 10/5 балів
- поточне оцінювання, max = 10/5 балів
- іспит, max = 40/24 бали

Триетапна перевірка програмних проєктів:
— автогрейдер
— перевірка якості коду
— співбесіда за кодом

Оцінки, що менші порогового рівня, не додаються.

Разом з іспитом необхідно набрати не менше 60 балів.

Кожен, хто вчиться вчасно та чесно, зазвичай без проблем складає іспит з першої спроби.

Загальні поняття програмування

Парадигми програмування

Парадигма програмування — сукупність ідей і понять, що визначають стиль написання комп'ютерних програм.

Загальні поняття програмування

Парадигми програмування

Парадигма програмування — сукупність ідей і понять, що визначають стиль написання комп'ютерних програм.

Декларативне програмування

для отримання результату
описуємо його властивості

тістечко має бути пухким та
солодким

Загальні поняття програмування

Парадигми програмування

Парадигма програмування — сукупність ідей і понять, що визначають стиль написання комп'ютерних програм.

Декларативне програмування	Імперативне програмування
для отримання результату описуємо його властивості	для отримання результату описуємо процес його отримання
тістечко має бути пухким та солодким	щоб отримати тістечко, візьміть яйця, борошно, цукор, ...

Загальні поняття програмування

Парадигми програмування

Парадигма програмування — сукупність ідей і понять, що визначають стиль написання комп'ютерних програм.

Декларативне програмування	Імперативне програмування
для отримання результату описуємо його властивості	для отримання результату описуємо процес його отримання
тістечко має бути пухким та солодким	щоб отримати тістечко, візьміть яйця, борошно, цукор, ...

Існують й інші парадигми.

Розглядаємо:

- імперативне програмування,
- елементи об'єктно-орієнтованого програмування.

Загальні поняття програмування

Програми

Існує багато означень того, що таке комп'ютерна програма, які доповнюють та уточнюють одне одного.

— це опис дій, які має виконати комп'ютер, щоб розв'язати деяку задачу.

— зображена в об'єктивній формі сукупність даних і команд, призначених для функціонування ЕОМ та інших комп'ютерних пристроїв з метою отримання певного результату, включаючи підготовчі матеріали, отримані під час розробки програми для ЕОМ, та породжувані нею аудіовізуальні відображення.

Програмування – діяльність, яка полягає у створенні програм.

Мета програмування – забезпечити, щоб замість людини ту чи іншу роботу виконував комп'ютер.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Дані зображують (=позначають) деякий зміст, або інформацію.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Дані зображують (=позначають) деякий зміст, або інформацію.

Комп'ютер оперує даними про реальні об'єкти або явища.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Дані зображують (=позначають) деякий зміст, або інформацію.

Комп'ютер оперує даними про реальні об'єкти або явища.

Зобразити об'єкт у вигляді даних – побудувати його модель.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Дані зображують (=позначають) деякий зміст, або інформацію.

Комп'ютер оперує даними про реальні об'єкти або явища.

Зобразити об'єкт у вигляді даних – побудувати його модель.

Модель – спрощене зображення об'єкта або явища, яке відображає його необхідні суттєві властивості та, можливо, співвідношення між ними.

Загальні поняття програмування

Дані, моделі

Дії комп'ютера пов'язані з обробкою даних.

Дані зображують (=позначають) деякий зміст, або інформацію.

Комп'ютер оперує даними про реальні об'єкти або явища.

Зобразити об'єкт у вигляді даних – побудувати його модель.

Модель – спрощене зображення об'єкта або явища, яке відображає його необхідні суттєві властивості та, можливо, співвідношення між ними.

Одна та сама сутність може мати безліч моделей.

Загальні поняття програмування

Мови

Дані є позначеннями властивостей об'єкта. Для правильного розуміння (відновлення позначеного змісту) необхідно знати, що саме позначають окремі його елементи.

Загальні поняття програмування

Мови

Дані є позначеннями властивостей об'єкта. Для правильного розуміння (відновлення позначеного змісту) необхідно знати, що саме позначають окремі його елементи.

Мова – система позначень деякого змісту; включає елементарні позначення, правила утворення складніших із простіших та правила, за якими зміст та позначення відповідають одне одному.

Загальні поняття програмування

Мови

Дані є позначеннями властивостей об'єкта. Для правильного розуміння (відновлення позначеного змісту) необхідно знати, що саме позначають окремі його елементи.

Мова – система позначень деякого змісту; включає елементарні позначення, правила утворення складніших із простіших та правила, за якими зміст та позначення відповідають одне одному.

Синтаксис мови визначається правилами утворення позначень.

Загальні поняття програмування

Мови

Дані є позначеннями властивостей об'єкта. Для правильного розуміння (відновлення позначеного змісту) необхідно знати, що саме позначають окремі його елементи.

Мова – система позначень деякого змісту; включає елементарні позначення, правила утворення складніших із простіших та правила, за якими зміст та позначення відповідають одне одному.

Синтаксис мови визначається правилами утворення позначень.

Семантика мови визначається правилами, які задають зміст позначень.

Загальні поняття програмування

Класифікація мов

Мови бувають: природні, штучні.

Загальні поняття програмування

Класифікація мов

Мови бувають: природні, штучні.

Штучні мови більш стандартизовані, не допускають можливості різного тлумачення змісту позначень.

Загальні поняття програмування

Класифікація мов

Мови бувають: природні, штучні.

Штучні мови більш стандартизовані, не допускають можливості різного тлумачення змісту позначень.

Мови програмування – штучні мови, призначені для запису програм.

Загальні поняття програмування

Класифікація мов

Мови бувають: природні, штучні.

Штучні мови більш стандартизовані, не допускають можливості різного тлумачення змісту позначень.

Мови програмування – штучні мови, призначені для запису програм.

Програма, записана мовою програмування, завжди має єдиний зміст :), але, можливо, не той, що мався на увазі програмістом :(((.

Загальні поняття програмування

Алгоритм, процес

Алгоритм – опис порядку дій, які треба виконати, щоб розв'язати деяку задачу.

У вузькому розумінні, програма – алгоритм, записаний мовою програмування.

Загальні поняття програмування

Алгоритм, процес

Алгоритм – опис порядку дій, які треба виконати, щоб розв'язати деяку задачу.
У вузькому розумінні, програма – алгоритм, записаний мовою програмування.

В алгоритмі:
– дії позначаються командами, або інструкціями (*statement*).

Загальні поняття програмування

Алгоритм, процес

Алгоритм – опис порядку дій, які треба виконати, щоб розв'язати деяку задачу.
У вузькому розумінні, програма – алгоритм, записаний мовою програмування.

В алгоритмі:
– дії позначаються командами, або інструкціями (*statement*).
– вказують вхідні (*input*), проміжні та вихідні (*output*) дані.

Загальні поняття програмування

Алгоритм, процес

Алгоритм – опис порядку дій, які треба виконати, щоб розв'язати деяку задачу.
У вузькому розумінні, програма – алгоритм, записаний мовою програмування.

В алгоритмі:
– дії позначаються командами, або інструкціями (*statement*).
– вказують вхідні (*input*), проміжні та вихідні (*output*) дані.

Процес – сукупність дій, що взаємозв'язані і взаємодіють та перетворюють вхідні дані на вихідні (ISO).

Загальні поняття програмування

Алгоритм, процес

Алгоритм – опис порядку дій, які треба виконати, щоб розв'язати деяку задачу.
У вузькому розумінні, програма – алгоритм, записаний мовою програмування.

В алгоритмі:
– дії позначаються командами, або інструкціями (*statement*).
– вказують вхідні (*input*), проміжні та вихідні (*output*) дані.

Процес – сукупність дій, що взаємозв'язані і взаємодіють та перетворюють вхідні дані на вихідні (ISO).

Один алгоритм може визначати більше одного процесу.

Загальні поняття програмування

Властивості алгоритмів

- зрозумілість
- результативність
- однозначність
- дискретність
- масовість
- виконуваність
- скінченність

Загальні поняття програмування

Властивості алгоритмів

- зрозумілість
- результативність
- однозначність
- дискретність
- масовість
- виконуваність
- скінченність

Гарно побудований алгоритм забезпечує закінчення дій з деяким результатом за будь-яких умов.

Загальні поняття програмування

Мови програмування

... – штучні мови, призначені для запису програм.

Загальні поняття програмування

Мови програмування

... – штучні мови, призначені для запису програм.

Машинна мова – це система команд, які може виконувати процесор.

Загальні поняття програмування

Мови програмування

... – штучні мови, призначені для запису програм.

Машинна мова – це система команд, які може виконувати процесор.

Машинні мови є мовами низького рівня, бо детально зображують дії комп'ютера.

Загальні поняття програмування

Мови програмування

... – штучні мови, призначені для запису програм.

Машинна мова – це система команд, які може виконувати процесор.

Машинні мови є мовами низького рівня, бо детально зображують дії комп'ютера.

Мови високого рівня (абстракції) вільні від машинних подробиць, ближчі до мов формул та природних мов.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Транслятори поділяються на компілятори та інтерпретатори.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Транслятори поділяються на компілятори та інтерпретатори.

Компілятор транслює (перекладає) текст програми, написаної мовою високого рівня, у програму на машинній мові (або дуже близькій до неї).

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Транслятори поділяються на компілятори та інтерпретатори.

Компілятор транслює (перекладає) текст програми, написаної мовою високого рівня, у програму на машинній мові (або дуже близькій до неї).

Компільованими мовами є C, C++.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Транслятори поділяються на компілятори та інтерпретатори.

Компілятор транслює (перекладає) текст програми, написаної мовою високого рівня, у програму на машинній мові (або дуже близькій до неї).

Компільованими мовами є C, C++.

Інтерпретатор покомандно або рядками аналізує та відразу виконує програму.

Мовні процесори

Компілятори та інтерпретатори

Мова C++ є мовою високого рівня.

Програми мовами високого рівня комп'ютер здатен виконати тільки після обробки мовними процесорами, тобто трансляторами.

Транслятори поділяються на компілятори та інтерпретатори.

Компілятор транслює (перекладає) текст програми, написаної мовою високого рівня, у програму на машинній мові (або дуже близькій до неї).

Компільованими мовами є C, C++.

Інтерпретатор покомандно або рядками аналізує та відразу виконує програму.

Інтерпретованими мовами є Python, JavaScript.

Мовні процесори

Різновиди трансляторів: буває й таке

— Програма перетворюється в програму на проміжній мові (доволі низькорівневій) – байт-код, який далі обробляється інтерпретатором.

Мовні процесори

Різновиди трансляторів: буває й таке

— Програма перетворюється в програму на проміжній мові (доволі низькорівневій) – байт-код, який далі обробляється інтерпретатором.

— За програмою утворюється виконуваний файл, що містить у собі інтерпретатор та байт-код.

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

—

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

— Для виконання підготовленого виконуваного файлу мати компілятор потреби нема.

—

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

— Для виконання підготовленого виконуваного файлу мати компілятор потреби нема.

—

— Для виконання інтерпретованої програми необхідно мати інтерпретатор.

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

— Для виконання підготовленого виконуваного файлу мати компілятор потреби нема.

— Для кожної ОС програму треба перекомпільовувати.

—

— Для виконання інтерпретованої програми необхідно мати інтерпретатор.

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

— Для виконання підготовленого виконуваного файлу мати компілятор потреби нема.

— Для кожної ОС програму треба перекомпільовувати.

—

— Для виконання інтерпретованої програми необхідно мати інтерпретатор.

— Одну ту саму інтерпретовану програму без додаткових дій можна виконувати під різними ОС (кросплатформеність). Розробники інтерпретаторів зазвичай відразу компілюють їх під різні платформи.

Мовні процесори

Компілятори versus інтерпретатори

— Відкомпільована програма виконується швидше, оскільки допоміжні перетворення були зроблені один раз та заздалегідь.

— Для виконання підготовленого виконуваного файлу мати компілятор потреби нема.

— Для кожної ОС програму треба перекомпільовувати.

— Щоб мати можливість будувати інтерпретатори, треба мати компілятор.

——

— Для виконання інтерпретованої програми необхідно мати інтерпретатор.

— Одну ту саму інтерпретовану програму без додаткових дій можна виконувати під різними ОС (кросплатформеність). Розробники інтерпретаторів зазвичай відразу компілюють їх під різні платформи.

Дуже короткий огляд історії мови C++

1970-і, початок: мова C (Денніс Рітчі)
1970-і, кінець: ООП, мова C++ (Бйорне Страуструп)
Нащадки: C#, Java

Дуже короткий огляд історії мови C++

1970-і, початок: мова C (Денніс Рітчі)
1970-і, кінець: ООП, мова C++ (Бйорне Страуструп)
Нащадки: C#, Java

Стандарти: C++17 (a must to know), C++20 (затверджений), C++23 (затверджений в кінці 2024-го року), C++26 (у розробці)

Дуже короткий огляд історії мови C++

1970-і, початок: мова C (Денніс Рітчі)
1970-і, кінець: ООП, мова C++ (Бйорне Страуструп)
Нащадки: C#, Java

Стандарти: C++17 (a must to know), C++20 (затверджений), C++23 (затверджений в кінці 2024-го року), C++26 (у розробці)

Підтримка нововведень компіляторами
https://en.cppreference.com/w/cpp/compiler_support

Корисні ресурсри

cppreference.com – URL: https://en.cppreference.com

Microsoft C/C++ Documentation | Microsoft Learn – URL: https://learn.microsoft.com/en-us/cpp/?view=msvc-170

Інтегроване середовище розробки, або IDE

IDE – система програмних засобів, що використовується для розробки програмного забезпечення.
Зв'язує між собою різні засоби, серед яких:

- текстовий редактор
- компілятор та компонувальник (для інтерпретованих мов інтерпретатор)
- налагоджувач
- засоби автоматизації контролю версій
- засоби перевірки стилю та форматування коду
- засоби реорганізації (рефакторингу) коду

Програмні засоби, необхідні для комфортного програмування на C++

IDE (достатньо одного з):
— Visual Studio (Community version)
компілятор MSVC можна встановити разом з IDE

— Visual Studio Code (окремо треба буде встановити компілятор та CMake)
— за бажанням можна використати інше сучасне IDE

компілятор: залежить від ОС
варіанти: MSVC, CLang, gcc/g++

Діяльності під час роботи з програмою

- “паперова” робота: аналіз задачі, розробка алгоритму, проєктування коду
- у текстовому редакторі набираємо текст програми
- компілюємо, збираємо виконуваний файл і тільки потім запускаємо отриманий виконуваний файл на виконання
- тестуємо та налагоджуємо
- передаємо замовнику та супроводжуємо (для реальних проєктів)

Перша програма мовою C++

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello, World!" << endl;
    return 0;
}
```

Програмування
Лекція 2. Лексичні елементи мови C++. Типи даних. Модель даних мови C++

09.09.2025

Алфавіт і словник мови C++

Деякі фази роботи транслятора

У роботі компілятора мови C++ можна виділити фазу роботи препроцесора та фази лексичного і синтаксичного аналізу (є й інші).

Алфавіт і словник мови C++

Деякі фази роботи транслятора

У роботі компілятора мови C++ можна виділити фазу роботи препроцесора та фази лексичного і синтаксичного аналізу (є й інші).

Лексема — послідовність символів вхідного тексту програми, що розглядається транслятором як неподільна і сама по собі має деякий зміст.

Алфавіт і словник мови C++

Деякі фази роботи транслятора

У роботі компілятора мови C++ можна виділити фазу роботи препроцесора та фази лексичного і синтаксичного аналізу (є й інші).

Лексема — послідовність символів вхідного тексту програми, що розглядається транслятором як неподільна і сама по собі має деякий зміст.

Задача лексичного аналізатора — розбити вхідний текст на лексеми.

Алфавіт і словник мови C++

Деякі фази роботи транслятора

У роботі компілятора мови C++ можна виділити фазу роботи препроцесора та фази лексичного і синтаксичного аналізу (є й інші).

Лексема — послідовність символів вхідного тексту програми, що розглядається транслятором як неподільна і сама по собі має деякий зміст.

Задача лексичного аналізатора — розбити вхідний текст на лексеми.

Отримана послідовність лексем передається синтаксичному аналізатору.

Алфавіт і словник мови C++

Деякі фази роботи транслятора

У роботі компілятора мови C++ можна виділити фазу роботи препроцесора та фази лексичного і синтаксичного аналізу (є й інші).

Лексема — послідовність символів вхідного тексту програми, що розглядається транслятором як неподільна і сама по собі має деякий зміст.

Задача лексичного аналізатора — розбити вхідний текст на лексеми.

Отримана послідовність лексем передається синтаксичному аналізатору.

Одна із задач синтаксичного аналізатора — перевірка синтаксичної правильності програми.

Алфавіт і словник мови C++

Алфавіт мови C++, деякі частини

- десяткові цифри 0, 1, ..., 9

Алфавіт і словник мови C++

Алфавіт мови C++, деякі частини

- десяткові цифри 0, 1, ..., 9
- літери (букви), зокрема:

Алфавіт і словник мови C++

Алфавіт мови C++, деякі частини

- десяткові цифри 0, 1, ..., 9
- літери (букви), зокрема:
  - латиниця A, B, ..., Z, a, b, ..., z

Алфавіт і словник мови C++

Алфавіт мови C++, деякі частини

- десяткові цифри 0, 1, ..., 9
- літери (букви), зокрема:
  - латиниця A, B, ..., Z, a, b, ..., z
  - _ (підкреслення)

Алфавіт і словник мови C++

Алфавіт мови C++, деякі частини

- десяткові цифри 0, 1, ..., 9
- літери (букви), зокрема:
  - латиниця A, B, ..., Z, a, b, ..., z
  - _ (підкреслення)
- знаки пунктуації , . + - (тощо
- пробіл та чотири білі символи
  зокрема символ нового рядка та табуляція

Словник мови C++

Види лексем

- літерали

  позначення значень

  12, 5.5, "asd"

Словник мови C++

Види лексем

- літерали

  позначення значень

  12, 5.5, "asd"

- ідентифікатори, або імена

  послідовність букв і цифр, що починається з букви

  program, Program, PROGRAM

  ✓ Регістр літер має значення.

  ✓ Імена з _ на початку зарезервовані компілятором.

Словник мови C++

Види лексем

- літерали

  позначення значень

  12, 5.5, "asd"

- ідентифікатори, або імена

  послідовність букв і цифр, що починається з букви

  program, Program, PROGRAM

  ✓ Регістр літер має значення.

  ✓ Імена з _ на початку зарезервовані компілятором.

- ключові, або зарезервовані, слова

  зарезервовані слова, що мають спеціальний зміст, визначений мовою програмування

  ✓ Використовувати ключові слова як власні ідентифікатори заборонено синтаксисом мови програмування.

  int, for

Словник мови C++

Види лексем

- оператори (operator), або знаки операцій
  позначення операцій, наприклад
  $+$, $<=$, sizeof
  ✓ Операції можуть позначатися символами або послідовностями символів.

Словник мови C++

Види лексем

- оператори (operator), або знаки операцій
  позначення операцій, наприклад
  +, <=, sizeof
  ✓ Операції можуть позначатися символами або послідовностями символів.
- знаки пунктуації, наприклад ( ) {

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

- лексичний аналізатор завжди намагається виділити найдовшу лексему

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

- лексичний аналізатор завжди намагається виділити найдовшу лексему
  qw = we *лексеми* qw, =, we

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

- лексичний аналізатор завжди намагається виділити найдовшу лексему

  qw = we *лексеми* qw, =, we

  for12+wq *лексеми* for12, +, wq

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

- лексичний аналізатор завжди намагається виділити найдовшу лексему
  qw = we *лексеми* qw, =, we
  for12+wq *лексеми* for12, +, wq
- якщо дві лексеми, записані поспіль, утворюють іншу лексему, то мають розділятися білими символами

Словник мови C++

Відокремлення лексем

У вхідному тексті лексеми можуть розділятися: операторами (крім ключових слів), знаками пунктуації, непорожніми послідовностями білих символів.

- лексичний аналізатор завжди намагається виділити найдовшу лексему
  qw = we *лексеми* qw, =, we
  for12+wq *лексеми* for12, +, wq
- якщо дві лексеми, записані поспіль, утворюють іншу лексему, то мають розділятися білими символами
- білі символи між лексемами одного рядка лексему не утворюють
  fun and sun *лексеми* fun, and, sun

Алфавіт і словник мови C++

Коментарі

Однорядковий коментар
— текст, записаний після двох скісних поспіль й до кінця рядка, включно з цими скісними

```
cout<<"Hello, World!"; //коментар
```

Багаторядковий коментар
— текст, записаний після /* й до */, включно з /* і */

```
/*  цей коментар
записано
в кількох рядках */
```

Алфавіт і словник мови C++

Коментарі

Однорядковий коментар
— текст, записаний після двох скісних поспіль й до кінця рядка, включно з цими скісними

```
cout<<"Hello, World!"; //коментар
```

Багаторядковий коментар
— текст, записаний після /* й до */, включно з /* і */

```
/*  цей коментар
записано
в кількох рядках */
```

Коментарі при компіляції ігноруються. Лексемами не є.

Алфавіт і словник мови C++

Коментарі

Однорядковий коментар
— текст, записаний після двох скісних поспіль й до кінця рядка, включно з цими скісними

```
cout<<"Hello, World!"; //коментар
```

Багаторядковий коментар
— текст, записаний після /* й до */, включно з /* і */

```
/*  цей коментар
записано
в кількох рядках */
```

Коментарі при компіляції ігноруються. Лексемами не є.

Корисність: пояснюють призначення коду або певні особливості, щоб полегшити роботу з кодом у майбутньому.

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Множина значень – носій типу, множина операцій – сигнатура.

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Множина значень – носій типу, множина операцій – сигнатура.

Типи даних підрозділяють на:

— скалярні

— складені (*composite*), або складні, або структуровані

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Множина значень – носій типу, множина операцій – сигнатура.

Типи даних підрозділяють на:
— скалярні
— складені (*composite*), або складні, або структуровані

Значення скалярних типів розглядаються як цілісні елементи, що не мають складових частин.

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Множина значень – носій типу, множина операцій – сигнатура.

Типи даних підрозділяють на:
— скалярні
— складені (*composite*), або складні, або структуровані

Значення скалярних типів розглядаються як цілісні елементи, що не мають складових частин.

Значення складених типів складаються з окремих компонентів.

Поняття типу даних

Тип даних — множина значень разом із множиною застосовних до них операцій.

Множина значень – носій типу, множина операцій – сигнатура.

Типи даних підрозділяють на:
— скалярні
— складені (*composite*), або складні, або структуровані

Значення скалярних типів розглядаються як цілісні елементи, що не мають складових частин.

Значення складених типів складаються з окремих компонентів.

Фундаментальні, або базові скалярні, типи — стандартні скалярні типи мови програмування.

Мінімальні відомості про фундаментальні типи

Типи, що зображують цілі значення

short, **int**, **long**, **long long**

Ціле кодується в додатковому коді. Кількість байтів залежить від компілятора.

Мінімальні відомості про фундаментальні типи

Типи, що зображують цілі значення

short, **int**, **long**, **long long**

Ціле кодується в додатковому коді. Кількість байтів залежить від компілятора.

unsigned short, **unsigned int**, **unsigned long**, **unsigned long long**

... зображують беззнакові цілі

Мінімальні відомості про фундаментальні типи

Типи, що зображують дійсні значення

float, **double**, **long double**

Мінімальні відомості про фундаментальні типи

Типи, що зображують дійсні значення

float, **double**, **long double**

Використовується зображення з плаваючою крапкою згідно стандарту IEEE754.
"Рідним" для процесора є зображення в типі **double** (займає 8 байтів.

Мінімальні відомості про фундаментальні типи

Типи, що зображують дійсні значення

float, **double**, **long double**

Використовується зображення з плаваючою крапкою згідно стандарту IEEE754.
"Рідним" для процесора є зображення в типі **double** (займає 8 байтів.

— Дійсні числа утворюють скінченну множину, що не є щільною.

Мінімальні відомості про фундаментальні типи

Типи, що зображують дійсні значення

float, **double**, **long double**

Використовується зображення з плаваючою крапкою згідно стандарту IEEE754.
"Рідним" для процесора є зображення в типі **double** (займає 8 байтів.

— Дійсні числа утворюють скінченну множину, що не є щільною.

— Арифметичні операції виконуються з наближенням до математичного результату.

Мінімальні відомості про фундаментальні типи

Типи, що зображують дійсні значення

float, **double**, **long double**

Використовується зображення з плаваючою крапкою згідно стандарту IEEE754.
"Рідним" для процесора є зображення в типі **double** (займає 8 байтів.

— Дійсні числа утворюють скінченну множину, що не є щільною.

— Арифметичні операції виконуються з наближенням до математичного результату.

— Множина дійсних чисел містить значення: $-\infty$, $+\infty$, -0, $+0$, та низку значень, що позначаються як **NaN**.

Мінімальні відомості про фундаментальні типи

Символьні типи

char, **signed char**, **unsigned char**

Типи даних

Внутрішнє зображення символів

У комп'ютері символи кодуються за допомогою 0 та 1.

Складається таблиця, що задає відповідність між символами та їх кодами, а далі коди зображуються в комп'ютері за допомогою 0 та 1.

Типи даних

Внутрішнє зображення символів

У комп'ютері символи кодуються за допомогою 0 та 1.

Складається таблиця, що задає відповідність між символами та їх кодами, а далі коди зображуються в комп'ютері за допомогою 0 та 1.

Історично перше кодування використовувало 7 бітів та кодувало символи числами від 0 до 127.

Американський стандартний код обміну інформацією, ASCII кодування ASCII, кодова таблиця ASCII

Типи даних

Властивості таблиці ASCII

- символам 0, 1, ..., 9 відповідають послідовні коди від 48 до 57
- символам A, B, ..., Z відповідають послідовні коди від 65 до 90
- символам a, b, ..., z відповідають послідовні коди від 97 до 122
- кодом пробілу є число 32 (0x20)
- кириличні символи відсутні

Типи даних

Розширена таблиця ASCII

За переходу на 8-бітове кодування (1 байт) до таблиці кодів додали символи з кодами 128..255. Отримали розширену ASCII.

Типи даних

Розширена таблиця ASCII

За переходу на 8-бітове кодування (1 байт) до таблиці кодів додали символи з кодами 128..255. Отримали розширену ASCII.

Її нижня частина — символи з кодами від 0 до 127, згадується як US-ASCII та визначається стандартом.

Типи даних

Розширена таблиця ASCII

За переходу на 8-бітове кодування (1 байт) до таблиці кодів додали символи з кодами 128..255. Отримали розширену ASCII.

Її нижня частина — символи з кодами від 0 до 127, згадується як US-ASCII та визначається стандартом.
Верхня частина містить символи з кодами від 128 до 255, намагається враховувати особливості національних алфавітів і є різною для різних кодувань, серед яких:

численні KOI-8, Windows-1251, 866 (DOS)

Типи даних

Розширена таблиця ASCII

За переходу на 8-бітове кодування (1 байт) до таблиці кодів додали символи з кодами 128..255. Отримали розширену ASCII.

Її нижня частина — символи з кодами від 0 до 127, згадується як US-ASCII та визначається стандартом.
Верхня частина містить символи з кодами від 128 до 255, намагається враховувати особливості національних алфавітів і є різною для різних кодувань, серед яких:

численні KOI-8, Windows-1251, 866 (DOS)

Використання різних кодувань не покращує переносимість коду та документів.

Багатобайтові кодування враховують національні особливості набагато краще.

У сучасному програмуванні тексти програм пишуть переважно в кодуванні UTF-8.

Мінімальні відомості про фундаментальні типи

Символьні типи

char, **signed char**, **unsigned char**

Числовий однобайтовий код символу записується у двійковій системі числення.

Мінімальні відомості про фундаментальні типи

Булівський тип

bool — булівський тип, або тип логічних значень чи просто логічний тип

Має тільки два можливих значення: **true** та **false** — «істина» та «хибність» відповідно.

Мінімальні відомості про фундаментальні типи

standard integer types (стандартні цілі типи):
short, **int**, **long**, **long long**,
unsigned short, **unsigned int**, **unsigned long**,
unsigned long long,
signed char, **unsigned char**

цілі (integral, або integer), типи:
до попередніх ще додаємо **char**, **bool**

арифметичні типи:
до попередніх ще додаємо **float**, **double**, **long double**

Літерали

Літерали позначають значення в текстах програм.

Можуть позначати значення різних типів: int, long, double, char та не тільки.

Літерали цілих типів

123, 45, 89 позначають значення типу **int**

Можна записувати в 10-й, 8-й, 16-й та 2-й системах числення та використовувати роздільники розрядів.

020, 0xFF, 0b0001'0000

- Запис, що починається з нуля, розглядається як вісімковий.

Літерали дійсних типів

12.34, .45, 123.4E-8, 1.324e1 позначають значення типу **double**

- У тексті програми зазвичай записуємо в 10-й системі числення.
- **Роздільником розрядів є крапка.**

Символьні літерали

Потрібний символ записується в одиничних лапках (апострофах). Лапки до значення символу не входять.

```
'a', '4', '+', '\t', '\n'
```

Деякі символи можемо записати тільки через керуючі послідовності.

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

\\	\
\n	символ переходу на новий рядок
\t	табуляція
\'	'
\"	"

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

\\	\
\n	символ переходу на новий рядок
\t	табуляція
\'	'
\"	"

Екранований символ — символ, перед яким стоїть \

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

\\	\
\n	символ переходу на новий рядок
\t	табуляція
\'	'
\"	"

Екранований символ — символ, перед яким стоїть \
Символ \ має екрануватись завжди.
Символ ' в символьних літералах має екрануватись завжди.

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

\\	\
\n	символ переходу на новий рядок
\t	табуляція
\'	'
\"	"

Екранований символ — символ, перед яким стоїть \
Символ \ має екрануватись завжди.
Символ ' в символьних літералах має екрануватись завжди.

Літерали

Символьні літерали: керуючі послідовності

Керуюча послідовність, або escape-послідовність, — послідовність символів, що починається з символу \ та позначає один символ.

\\	\
\n	символ переходу на новий рядок
\t	табуляція
\'	'
\"	"

Екранований символ — символ, перед яким стоїть \
Символ \ має екрануватись завжди.
Символ ' в символьних літералах має екрануватись завжди.

Літерали

Рядкові літерали

```
"lite\"ral", "c:\\windows"
```

задають рядки, точніше C-рядки

```
lite"ral, c:\windows
```

З точки зору математики
Рядок (string) — послідовність символів.

Значеннями наведених літералів є рядки, утворені відповідними послідовностями символів. Але в пам'яті зберігається послідовність символів, на кінець якої додано символ '\0' (символ з кодом 0).

Змінні

Поняття змінної в інформатиці

Змінна величина (Декарт) — модель реального або уявного об'єкта чи його окрема характеристика, що може перебувати в різних станах.

У книжках з програмування та інформатики можна прочитати багато означень того, що таке змінна в програмуванні. Впливає і парадигма, і мова програмування, і рік видання.

Змінні

Властивості змінних

Властивості змінних можна розглядати:

Змінні

Властивості змінних

Властивості змінних можна розглядати:

- з точки зору тексту програми
  У тексті програми змінні позначаються <u>іменами</u> (ідентифікаторами).

Змінні

Властивості змінних

Властивості змінних можна розглядати:

- з точки зору тексту програми
  У тексті програми змінні позначаються <u>іменами</u> (ідентифікаторами).
- з точки зору процесу виконання програми
  В імперативних мовах програмування під час виконання змінній відповідає певна сутність, що зберігає її <u>значення</u>.

Змінні

Властивості змінних

Властивості змінних можна розглядати:

- з точки зору тексту програми
  У тексті програми змінні позначаються <u>іменами</u> (ідентифікаторами).
- з точки зору процесу виконання програми
  В імперативних мовах програмування під час виконання змінній відповідає певна сутність, що зберігає її <u>значення</u>.

Над змінними можна виконувати операції:

- встановлення значення змінної (присвоювання)
- добування значення змінної (розіменування імені змінної)

Змінні

Властивості змінних: тип змінної

Змінна може мати тип, що задає формат зберігання її значення, методи маніпуляції та трактовки вмісту.

Змінні

Властивості змінних: тип змінної

Змінна може мати тип, що задає формат зберігання її значення, методи маніпуляції та трактовки вмісту.

Типізовані мови: кожна змінна має тип.

Безтиповими є машинні мови.

Змінні

Властивості змінних: тип змінної

Змінна може мати тип, що задає формат зберігання її значення, методи маніпуляції та трактовки вмісту.

Типізовані мови: кожна змінна має тип.

Безтиповими є машинні мови.

Статична типізація: тип змінної може бути визначений на етапі трансляції (до виконання).
Динамічна типізація: тип змінної визначається під час виконання.

Змінні

Властивості змінних: тип змінної

Змінна може мати тип, що задає формат зберігання її значення, методи маніпуляції та трактовки вмісту.

Типізовані мови: кожна змінна має тип.

Безтиповими є машинні мови.

Статична типізація: тип змінної може бути визначений на етапі трансляції (до виконання).
Динамічна типізація: тип змінної визначається під час виконання.

- Мова C++ є статично типізованою.

Змінні

Змінні мови C++

У тексті програми змінна асоціюється зі своїм іменем та позначає певну сутність, яка матеріалізується під час виконання програми у вигляді ділянки оперативної пам'яті, що зберігає значення змінної.

Ім'я змінної — ідентифікатор, що позначає змінну в програмі.
Значення змінної — (під час виконання) вміст ділянки пам'яті, в якій зберігається змінна.
Адреса змінної — (під час виконання) адреса першого байту ділянки пам'яті, в якій зберігається змінна.

Змінні

Змінні мови C++

У тексті програми змінна асоціюється зі своїм іменем та позначає певну сутність, яка матеріалізується під час виконання програми у вигляді ділянки оперативної пам'яті, що зберігає значення змінної.

Ім'я змінної — ідентифікатор, що позначає змінну в програмі.
Значення змінної — (під час виконання) вміст ділянки пам'яті, в якій зберігається змінна.
Адреса змінної — (під час виконання) адреса першого байту ділянки пам'яті, в якій зберігається змінна.

- Змінна завжди займає неперервну ділянку пам'яті.

Змінні

Змінні мови C++

У тексті програми змінна асоціюється зі своїм іменем та позначає певну сутність, яка матеріалізується під час виконання програми у вигляді ділянки оперативної пам'яті, що зберігає значення змінної.

Ім'я змінної — ідентифікатор, що позначає змінну в програмі.
Значення змінної — (під час виконання) вміст ділянки пам'яті, в якій зберігається змінна.
Адреса змінної — (під час виконання) адреса першого байту ділянки пам'яті, в якій зберігається змінна.

- Змінна завжди займає неперервну ділянку пам'яті.

Ще для змінної можна розглядати час існування (lifetime) та клас пам'яті (storage duration), який визначає спосіб керування часом існування.

Змінні

Інструкція оголошення [імені] змінної

<позначення типу> <ім'я змінної>;
<позначення типу> <перелік імен змінних через кому;

double radius; // Змінним слід давати змістовні імена!
int n, m;

Оголошення – повідомляє компілятору про певну сутність.
Означення – означає певну сутність.
Означення змінної задає створення змінної: виділяється ділянка пам'яті, де буде зберігатися значення змінної. У прикладах оголошення одночасно є означенням.

- Кожна використовувана змінна має бути оголошена та означена.
- Якщо змінна не отримала початкове значення, то розіменовувати її не можна.

Якщо ми явно не передбачимо, то початкове значення під час створення змінна може не отримати.

Змінні

Задаємо значення змінної

- значення можна задати під час означення
- значення можна присвоїти

Змінні

Ініціалізація змінної

... встановлення початкового значення під час її означення

```
int n = 10;
int m = k = 13;
```

Змінні

Присвоювання значення змінній

... у найпростішому випадку можна задати так:
<ім'я змінної> = <вираз>;

```
int a, b;
a = 10;
b = a;
```

Змінні

Введення значення в змінну

cin >> <ім'я змінної>;

```
int a, b;
cin >> a; cin >> b;
cin >> a >> b;
```

Білі символи вхідної послідовності символів є роздільниками лексем для операції добування значення з потоку (позначається оператором >>).

Це приклад ПОГАНОГО коду, бо користувачу не повідомляють, що від нього хочуть.

І в ньому є ще друга проблема (спробуйте ввести щось нечислове).

• Перед введенням не варто залишати змінну без початкового значення, якщо обробка помилок не додана.

Програмування
Лекція 3. Вирази. Операції. Математичні функції

16/09/2025

Вирази та їх складові

Математична формула $(a \cdot h)/2$

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Імена позначають змінні величини – об'єкти чи їх окремі характеристики, які можуть перебувати в різних станах.

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Імена позначають змінні величини – об'єкти чи їх окремі характеристики, які можуть перебувати в різних станах.

Операції (у програмуванні) – обчислювальні дії
— позначаються операторами (знаки, можливі імена)
— застосовуються до операндів (значень)

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Імена позначають змінні величини – об'єкти чи їх окремі характеристики, які можуть перебувати в різних станах.

Операції (у програмуванні) – обчислювальні дії
— позначаються операторами (знаки, можливі імена)
— застосовуються до операндів (значень)

Значення позначаються іменами та літералами.

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Імена позначають змінні величини – об'єкти чи їх окремі характеристики, які можуть перебувати в різних станах.

Операції (у програмуванні) – обчислювальні дії
— позначаються операторами (знаки, можливі імена)
— застосовуються до операндів (значень)

Значення позначаються іменами та літералами.

Дужки задають порядок застосування операцій.

Вирази та їх складові

Математична формула $(a \cdot h)/2$ утворює вираз, що складається з імен, позначень операцій, констант, дужок.

Імена позначають змінні величини – об'єкти чи їх окремі характеристики, які можуть перебувати в різних станах.

Операції (у програмуванні) – обчислювальні дії
— позначаються операторами (знаки, можливі імена)
— застосовуються до операндів (значень)

Значення позначаються іменами та літералами.

Дужки задають порядок застосування операцій.

Вираз задає:
— послідовність застосування операцій до значень
— значення, що є результатом цієї послідовності.

Обчислення виразу — послідовність застосувань операцій.

Оператори та операції

Арність

Оператори та операції класифікуються за кількістю аргументів (арністю).

Оператори та операції

Арність

Оператори та операції класифікуються за кількістю аргументів (арністю).

Найчастіше зустрічаються бінарні, або двомісні, наприклад, множення $*$.

Оператори та операції

Арність

Оператори та операції класифікуються за кількістю аргументів (арністю).

Найчастіше зустрічаються бінарні, або двомісні, наприклад, множення $*$.

Також розповсюджені унарні, або одномісні, наприклад, унарний плюс $+$ та унарний мінус $-$.

Оператори та операції

Арність

Оператори та операції класифікуються за кількістю аргументів (арністю).

Найчастіше зустрічаються бінарні, або двомісні, наприклад, множення $*$.

Також розповсюджені унарні, або одномісні, наприклад, унарний плюс $+$ та унарний мінус $-$.

Корисність унарного мінусу:
— для дійсного x вираз $-x$ обчислюється швидше ніж $(-1) * x$
— значення -13 у тексті програми задається як -13, яке розглядається компілятором як застосування унарного мінусу до 13

Оператори та операції

Арність

Оператори та операції класифікуються за кількістю аргументів (арністю).

Найчастіше зустрічаються бінарні, або двомісні, наприклад, множення $*$.

Також розповсюджені унарні, або одномісні, наприклад, унарний плюс $+$ та унарний мінус $-$.

Корисність унарного мінусу:
— для дійсного x вираз $-x$ обчислюється швидше ніж $(-1) * x$
— значення -13 у тексті програми задається як -13, яке розглядається компілятором як застосування унарного мінусу до 13

Існує тернарна операція (з трьома аргументами).

Оператори мови C++

Бінарні оператори (деякі)

операція	оператор
додавання	$+$
віднімання	$-$
множення	$*$
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Оператори мови C++

Бінарні оператори (деякі)

операція	оператор
додавання	$+$
віднімання	$-$
множення	$*$
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

- Операція визначена над аргументами одного типу і результатом її обчислення є значення цього ж типу.

Оператори мови C++

Бінарні оператори (деякі)

операція	оператор
додавання	$+$
віднімання	$-$
множення	$*$
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

• Операція визначена над аргументами одного типу і результатом її обчислення є значення цього ж типу.

аргументи та результат матимуть тип **int**: $2 + 2$, $2 * 6$
аргументи та результат матимуть тип **double**: $1.2 + 2.8$, $2.0 * 0.5$

Оператори мови C++

Ділення

операція	оператор
ділення	/
цілочисельне ділення	/
остача від цілочисельного ділення	%

Оператори мови C++

Ділення

операція	оператор
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Аргументи ділення дійсні — звичайне ділення, результат є дійсним.

Оператори мови C++

Ділення

операція	оператор
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Аргументи ділення дійсні — звичайне ділення, результат є дійсним.

аргументи типу **double**: $3.0/2.0$, $10.0/2.0$, $1.0/2.0$
результат типу **double**: 1.5, 5.0, 0.5

Оператори мови C++

Ділення

операція	оператор
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Аргументи ділення дійсні — звичайне ділення, результат є дійсним.

аргументи типу **double**: $3.0/2.0$, $10.0/2.0$, $1.0/2.0$
результат типу **double**: 1.5, 5.0, 0.5

Аргументи ділення цілі — цілочисельне ділення, результат є цілим (отримується відкиданням дробових розрядів)

Оператори мови C++

Ділення

операція	оператор
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Аргументи ділення дійсні — звичайне ділення, результат є дійсним.

аргументи типу **double**: $3.0/2.0$, $10.0/2.0$, $1.0/2.0$
результат типу **double**: 1.5, 5.0, 0.5

Аргументи ділення цілі — цілочисельне ділення, результат є цілим (отримується відкиданням дробових розрядів)

аргументи типу **int**: $3/2$, $10/2$, $1/2$, $(-7)/5$,
результат типу **int**: 1, 5, 0, -1

Оператори мови C++

Ділення

операція	оператор
ділення	$/$
цілочисельне ділення	$/$
остача від цілочисельного ділення	$\%$

Аргументи ділення цілі — цілочисельне ділення, результат є цілим (отримується відкиданням дробових розрядів)

аргументи типу **int**: $3/2$, $10/2$, $1/2$, $(-7)/5$,
результат типу **int**: 1, 5, 0, -1

Операція взяття остачі від цілочисельного ділення застосовна тільки до цілих (integral) аргументів.

Властивості операції цілочисельного ділення:
$(x/y) * y + (x\%y) = x$

$7\%5 = 2$, $(-7)\%5 = -2$, $7\%(-5) = 2$, $(-7)\%(-5) = -2$

Оператори мови C++

Оператор **sizeof**

Оператор **sizeof** застосовується до типів та виразів.

sizeof(int) — кількість байтів, яку в пам'яті займатиме значення типу **int**

дужки навколо типу обов'язкові

Оператори мови C++

Оператор **sizeof**

Оператор **sizeof** застосовується до типів та виразів.

sizeof(int) — кількість байтів, яку в пам'яті займатиме значення типу **int**

дужки навколо типу обов'язкові

sizeof(id) — кількість байтів, яку в пам'яті займає
sizeof id змінна **id**

sizeof(expr) — кількість байтів, які в пам'яті займатиме результат обчислення виразу **expr**

Оператори мови C++

Оператор **sizeof**

Оператор **sizeof** застосовується до типів та виразів.

sizeof(int) — кількість байтів, яку в пам'яті займатиме значення типу **int**

дужки навколо типу обов'язкові

sizeof(id) — кількість байтів, яку в пам'яті займає
sizeof id змінна **id**

sizeof(expr) — кількість байтів, які в пам'яті займатиме результат обчислення виразу **expr**

```
long n;
...
sizeof(n)
...
sizeof n
```

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 - \curvearrowleft 2 - \curvearrowleft 3 - 4$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 -^{\curvearrowleft} 2 -^{\curvearrowleft} 3 - 4 \qquad ((1 - 2) - 3) - 4$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 - \curvearrowleft 2 - \curvearrowleft 3 - 4 \qquad ((1 - 2) - 3) - 4$

Розглянуті раніше бінарні оператори мови C++ мають властивість лівобічного зв'язування.

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо) за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 - \curvearrowleft 2 - \curvearrowleft 3 - 4 \qquad ((1 - 2) - 3) - 4$

Розглянуті раніше бінарні оператори мови C++ мають властивість лівобічного зв'язування.

правобічне зв'язування: (дужки розставляються справа наліво) за рівнопріоритетних операторів операнд зв'язується з оператором праворуч

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо)
за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 - \curvearrowleft 2 - \curvearrowleft 3 - 4 \qquad ((1 - 2) - 3) - 4$

Розглянуті раніше бінарні оператори мови C++ мають властивість лівобічного зв'язування.

правобічне зв'язування: (дужки розставляються справа наліво)
за рівнопріоритетних операторів операнд зв'язується з оператором праворуч
нехай $\Diamond$ позначає деякий оператор з правобічним зв'язуванням
$x \Diamond y \Diamond 3$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо)
за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 - \curvearrowleft 2 - \curvearrowleft 3 - 4 \qquad ((1 - 2) - 3) - 4$

Розглянуті раніше бінарні оператори мови C++ мають властивість лівобічного зв'язування.

правобічне зв'язування: (дужки розставляються справа наліво)
за рівнопріоритетних операторів операнд зв'язується з оператором праворуч
нехай $\Diamond$ позначає деякий оператор з правобічним зв'язуванням

$x \Diamond y \Diamond 3 \qquad x \Diamond y \curvearrowright \Diamond 3$

Пріоритет операторів

Пріоритет, або старшинство (*precedence*), операторів дозволяє не записувати частину дужок.

$1 + 2 * 3 \qquad 1 + 2 \curvearrowright * 3 \qquad 1 + (2 * 3)$

лівобічне зв'язування: (дужки розставляються зліва направо)
за рівнопріоритетних операторів операнд зв'язується з оператором ліворуч

$1 - 2 - 3 - 4 \qquad 1 -^{\curvearrowleft} 2 -^{\curvearrowleft} 3 - 4 \qquad ((1 - 2) - 3) - 4$

Розглянуті раніше бінарні оператори мови C++ мають властивість лівобічного зв'язування.

правобічне зв'язування: (дужки розставляються справа наліво)
за рівнопріоритетних операторів операнд зв'язується з оператором праворуч
нехай $\Diamond$ позначає деякий оператор з правобічним зв'язуванням

$x \Diamond y \Diamond 3 \qquad x \Diamond y \curvearrowright \Diamond 3 \qquad x \Diamond (y \Diamond 3)$

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4 + 7/5$	$4 + (7/5)$	5

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4 + 7/5$	$4 + (7/5)$	5
$2 * 7\%8$	$(2 * 7)\%8$	6

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4 + 7/5$	$4 + (7/5)$	5
$2 * 7\%8$	$(2 * 7)\%8$	6
$4 - 3 - 2$	$(4 - 3) - 2$	-1

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4+7/5$	$4+(7/5)$	5
$2*7\%8$	$(2*7)\%8$	6
$4-3-2$	$(4-3)-2$	-1
$2*3/6$	$(2*3)/6$	1

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4+7/5$	$4+(7/5)$	5
$2*7\%8$	$(2*7)\%8$	6
$4-3-2$	$(4-3)-2$	-1
$2*3/6$	$(2*3)/6$	1
$2/6*3$	$(2/6)*3$	0

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4+7/5$	$4+(7/5)$	5
$2*7\%8$	$(2*7)\%8$	6
$4-3-2$	$(4-3)-2$	-1
$2*3/6$	$(2*3)/6$	1
$2/6*3$	$(2/6)*3$	0

Пріоритетами операторів зловживати не слід!

Таблиця пріоритетів (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$

Вираз	Розстановка дужок	Результат
$4+7/5$	$4+(7/5)$	5
$2*7\%8$	$(2*7)\%8$	6
$4-3-2$	$(4-3)-2$	-1
$2*3/6$	$(2*3)/6$	1
$2/6*3$	$(2/6)*3$	0

Пріоритетами операторів зловживати не слід!

Для покращення сприйняття коду можна використовувати не тільки дужки, але і пробіли: `x*y + y*z`

Особливості обчислення виразів у мові C++

Мова C++ у загальному випадку не регламентує порядок обчислення операндів та підвиразів.

$(expr1 + expr2) * expr3$

Тут гарантується тільки те, що

- $expr1$, $expr2$ будуть обчислені до обчислення операції додавання
- результат додавання та $expr3$ будуть обчисленні до обчислення операції множення

Особливості обчислення виразів у мові C++

Мова C++ у загальному випадку не регламентує порядок обчислення операндів та підвиразів.

$(expr1 + expr2) * expr3$

Тут гарантується тільки те, що

- $expr1$, $expr2$ будуть обчислені до обчислення операції додавання
- результат додавання та $expr3$ будуть обчисленні до обчислення операції множення

Для сучасних архітектур процесорів в якому порядку будуть обчислюватись $expr1$, $expr2$ та $expr3$ точно не скаже навіть компілятор.

Особливості обчислення виразів у мові C++

Мова C++ у загальному випадку не регламентує порядок обчислення операндів та підвиразів.

$(expr1 + expr2) * expr3$
Тут гарантується тільки те, що

- $expr1$, $expr2$ будуть обчислені до обчислення операції додавання
- результат додавання та $expr3$ будуть обчисленні до обчислення операції множення

Для сучасних архітектур процесорів в якому порядку будуть обчислюватись $expr1$, $expr2$ та $expr3$ точно не скаже навіть компілятор.

Для виразів $expr1 << expr2$ та $expr1 >> expr2$
мова C++ (починаючи з стандарту C++17) гарантує, що спочатку буде обчислено $expr1$, потім $expr2$ і потім значення виразу в цілому.

Оператор присвоювання, найпростіший випадок

<ім'я змінної> = <вираз>

Змінній буде присвоєно значення виразу, результатом обчислення цього виразу в деякому наближенні до правди можна вважати присвоєне значення.

⇒ Можливі ланцюгові присвоювання.

```
int n, m;
n = 25;
n = m = 10; // ланцюгове присвоювання
```

Оператор = має властивість правобічного зв'язування, тобто **x=y=z** обчислюється як **x=(y=z)**.

Оператор присвоювання, найпростіший випадок

<ім'я змінної> = <вираз>

Змінній буде присвоєно значення виразу, результатом обчислення цього виразу в деякому наближенні до правди можна вважати присвоєне значення.

⇒ Можливі ланцюгові присвоювання.

```
int n, m;
n = 25;
n = m = 10; // ланцюгове присвоювання
```

Оператор = має властивість правобічного зв'язування, тобто **x=y=z** обчислюється як **x=(y=z)**.

```
n = m = 10;        n = (m = 10);
```

```
m = 10;
n = m;
```

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$$expr1 = expr2$$

```
int n, m = 13;
n = m;
```

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$$expr1 = expr2$$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$$expr1 = expr2$$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

- ділянку пам'яті, де зберігається змінна
- значення, що зберігається в цій ділянці пам'яті

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$$expr1 = expr2$$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

- ділянку пам'яті, де зберігається змінна
- значення, що зберігається в цій ділянці пам'яті

Якщо вираз задає ділянку пам'яті, то він є l-виразом (lvalue).
l-виразами для цього прикладу є: **n**, **m**
l-виразами не є: **n+m**, **2*n**, **13** (це r-вирази)

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$expr1 = expr2$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

- ділянку пам'яті, де зберігається змінна
- значення, що зберігається в цій ділянці пам'яті

Якщо вираз задає ділянку пам'яті, то він є l-виразом (lvalue).
l-виразами для цього прикладу є: **n**, **m**
l-виразами не є: **n+m**, **2*n**, **13** (це r-вирази)

- Ліворуч від оператора = має бути вираз, який є l-виразом.

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$expr1 = expr2$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

- ділянку пам'яті, де зберігається змінна
- значення, що зберігається в цій ділянці пам'яті

Якщо вираз задає ділянку пам'яті, то він є l-виразом (lvalue).
l-виразами для цього прикладу є: **n**, **m**
l-виразами не є: **n+m**, **2*n**, **13** (це r-вирази)

- Ліворуч від оператора = має бути вираз, який є l-виразом.
- Вираз присвоювання є l-виразом.

Оператор присвоювання, шукаємо правду

Загальний вигляд оператора присвоювання

$expr1 = expr2$

```
int n, m = 13;
n = m;
```

Ім'я змінної є виразом. Що задає цей вираз?

- ділянку пам'яті, де зберігається змінна
- значення, що зберігається в цій ділянці пам'яті

Якщо вираз задає ділянку пам'яті, то він є l-виразом (lvalue).
l-виразами для цього прикладу є: **n**, **m**
l-виразами не є: **n+m**, **2*n**, **13** (це r-вирази)

• Ліворуч від оператора $=$ має бути вираз, який є l-виразом.

• Вираз присвоювання є l-виразом.

• Порядок обчислення: $expr2$, потім $expr1$, потім присвоювання.

Оператор присвоювання, шукаємо правду далі

Інструкція виразу

```
int n, m = 13;
n = m;  // ??? у запису виразу присвоювання нема ;
```

Оператор присвоювання, шукаємо правду далі

Інструкція виразу

```
int n, m = 13;
n = m;  // ??? у запису виразу присвоювання нема ;
```

<вираз>; — інструкція виразу

Інструкція виразу задає обчислення виразу, але отримане значення не використовує (discard).

Складені (compound) присвоювання

<expr1> += <expr2> $+$, $-$, $*$, $/$, $\%$ та деякі інші

```
int n = 13;
n += 7;
```

Складені (compound) присвоювання

<expr1> += <expr2> $+, -, *, /, \%$ та деякі інші

```
int n = 13;
n += 7;
```

Для вбудованих скалярних типів

<expr1> += <expr2>

обчислюється як

<expr1> = <expr1> + <expr2>

але значення виразу <expr1> обчислюється один раз

Таблиця пріоритетів, оновлена (за спаданням)

унарні $+$,$-$, **sizeof**
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
$<<$, $>>$
$=$, $+=$, $*=$, ... (обчислюються справа наліво)

Перетворення типів

Синтаксичне оформлення явного перетворення типу

Перетворення типу, або приведення до типу, — утворення значення одного типу за значенням іншого.

Може бути явним (явно задаємо в коді) та неявним (виконується без явної вказівки).

Перетворення типів

Синтаксичне оформлення явного перетворення типу

Перетворення типу, або приведення до типу, — утворення значення одного типу за значенням іншого.

Може бути явним (явно задаємо в коді) та неявним (виконується без явної вказівки).

Способи синтаксичного оформлення, на прикладі:
`double x = 3.14;`

- `(int)x`
- `int(x)`
- `static_cast<int>(x)`

Перетворення типів

Семантика перетворень

- Цілі числа можна перетворювати на дійсні.
  • Якщо значення не може бути зображено точно, то зображується наближено.
  • Якщо значення за межами допустимого діапазону, то це невизначена поведінка (undefined behaviour).

  `13` буде перетворене на `13.0` без втрати значення.

Перетворення типів

Семантика перетворень

- Цілі числа можна перетворювати на дійсні.
  • Якщо значення не може бути зображено точно, то зображується наближено.
  • Якщо значення за межами допустимого діапазону, то це невизначена поведінка (undefined behaviour).

  `13` буде перетворене на `13.0` без втрати значення.
- Дійсні числа можна перетворювати на цілі.
  • Перетворення відбувається шляхом відкидання дробових розрядів.
  • Якщо перетворене значення за межами допустимого діапазону, то це невизначена поведінка.

  `3.5`, `-3.5`, `3.8` будуть перетворені на `3`, `-3`, `3` відповідно.

Перетворення типів

Семантика перетворень

- Числа можна перетворювати на тип **bool**.
  - Ненульові значення перетворюються на **true**.
  - Нульові значення перетворюються на **false**.

  `3.5`, `-3.5`, `5` будуть перетворені на `true`, `0.0`, `0` на `false`.

Перетворення типів

Семантика перетворень

- Числа можна перетворювати на тип **bool**.
  - Ненульові значення перетворюються на **true**.
  - Нульові значення перетворюються на **false**.

  `3.5`, `-3.5`, `5` будуть перетворені на `true`, `0.0`, `0` на `false`.
- Логічні значення можна перетворювати на числові.
  - **true** перетворюється на 1 або 1.0 (залежно від типу).
  - **false** перетворюється на 0 або 0.0 (залежно від типу).

Перетворення типів

Семантика перетворень

- Значення цілого типу можна перетворювати на інший цілий тип.
  • Якщо значення не може бути зображено точно, то зображується зі збереженням остачі при діленні на 2^{8N}, де N – розмір у байтах типу результату.

  $\Rightarrow$ При перетворенні значення між типами **int** та **unsigned** результат перетворення кодується тією самою послідовністю бітів.

Перетворення типів

Семантика перетворень

- Значення цілого типу можна перетворювати на інший цілий тип.
  • Якщо значення не може бути зображено точно, то зображується зі збереженням остачі при діленні на 2^{8N}, де N – розмір у байтах типу результату.

  $\Rightarrow$ При перетворенні значення між типами **int** та **unsigned** результат перетворення кодується тією самою послідовністю бітів.
- Значення дійсного типу можна перетворювати на інший дійсний тип.
  • Якщо значення не може бути зображено точно, то зображується наближено.
  • Якщо значення за межами допустимого діапазону, то це невизначена поведінка.

Неявні перетворення типу

$$1 + 3.5$$

Операція має застосовуватись до аргументів одного типу. Тут аргументи різних типів. Що саме робить можливим застосування операції у цьому прикладі?

Неявні перетворення типу

$1 + 3.5$

Операція має застосовуватись до аргументів одного типу. Тут аргументи різних типів. Що саме робить можливим застосування операції у цьому прикладі?

— неявне перетворення типу

Неявні перетворення типу

$1 + 3.5$

Операція має застосовуватись до аргументів одного типу. Тут аргументи різних типів. Що саме робить можливим застосування операції у цьому прикладі?

— неявне перетворення типу

Якщо більш точно, то у випадку застосування арифметичних операторів (і порівнянь) до арифметичних типів виконуються так звані **звичайні (usual) арифметичні перетворення**, які додатково ще визначають тип результату. Після перетворень типу виконується обчислення операції.

Перетворення типів

Ранжування цілих типів (Integer conversion ranks) та “інтегральне просування”

Від найбільшого до найменшого

- long long, unsigned long long
- long, unsigned long
- int, unsigned int
- short, unsigned short
- char, unsigned char, signed char
- bool

Перетворення типів

Ранжування цілих типів (Integer conversion ranks) та “інтегральне просування”

Від найбільшого до найменшого

- long long, unsigned long long
- long, unsigned long
- int, unsigned int
- short, unsigned short
- char, unsigned char, signed char
- bool

Integral promotions перетворюють значення типів, менших за **int**, на **int** або **unsigned** (якщо тип **int** не може вмістити всі значення перетворюваного типу).

Звичайні (usual) арифметичні перетворення

Випадок: Серед типів операндів є дійсні

Вибирається перший з типів

long double

double

float

що наявний серед операндів,

виконується перетворення до нього,

на цьому перетворення типів завершені.

Звичайні (usual) арифметичні перетворення

Випадок: Серед типів операндів є дійсні

Вибирається перший з типів
long double
double
float
що наявний серед операндів,
виконується перетворення до нього,
на цьому перетворення типів завершені.

int, double $\rightarrow$ double
double, float $\rightarrow$ double

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів

Над операндами виконується “інтегральне просування”.

$\Rightarrow$ результат арифметичної операції не може мати тип менший за int, unsigned int

Якщо типи однакові, то кінець перетворень.

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів

Над операндами виконується “інтегральне просування”.

$\Rightarrow$ результат арифметичної операції не може мати тип менший за int, unsigned int

Якщо типи однакові, то кінець перетворень.

int, char $\rightarrow$ int
bool, char $\rightarrow$ int

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів, після інтегрального просування

Якщо типи однакової знаковості, то виконується перетворення меншого за рангом типу до більшого

int, long $\rightarrow$ long

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів, після інтегрального просування

Якщо типи однакової знаковості, то виконується перетворення меншого за рангом типу до більшого

int, long $\rightarrow$ long

Інакше якщо ранг беззнакового типу більше або дорівнює, то виконується перетворення до нього.

long, unsigned long $\rightarrow$ unsigned long

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів, після інтегрального просування

Якщо типи однакової знаковості, то виконується перетворення меншого за рангом типу до більшого

int, long $\rightarrow$ long

Інакше якщо ранг беззнакового типу більше або дорівнює, то виконується перетворення до нього.

long, unsigned long $\rightarrow$ unsigned long

Інакше якщо знаковий тип може вмістити всі значення беззнакового типу, то виконується перетворення до нього.

long long, unsigned int $\rightarrow$ long long

(за умови, що long long використовує більше байтів)

Звичайні (usual) арифметичні перетворення

Випадок: Операнди цілих типів, після інтегрального просування

Якщо типи однакової знаковості, то виконується перетворення меншого за рангом типу до більшого

int, long $\rightarrow$ long

Інакше якщо ранг беззнакового типу більше або дорівнює, то виконується перетворення до нього.

long, unsigned long $\rightarrow$ unsigned long

Інакше якщо знаковий тип може вмістити всі значення беззнакового типу, то виконується перетворення до нього.

long long, unsigned int $\rightarrow$ long long

(за умови, що long long використовує більше байтів)

Інакше виконується перетворення до беззнакового типу, що за розміром відповідає знаковому.

(непорівнювані типи різної знаковості та однакового розміру)

Сумісність типів

Сумісність типів в арифметичних виразах — можливість сумісного використання операндів цих типів.

Сумісність типів

Сумісність типів в арифметичних виразах — можливість сумісного використання операндів цих типів.

Сумісність типів за присвоюванням — можливість присвоювання змінним одного типу значень іншого типу.

Усі арифметичні типи сумісні за присвоюванням та у виразах.

При присвоюванні змінній одного арифметичного типу T1 значення іншого арифметичного типу T2 відбувається приведення цього значення до типу T1.

```
int n;
n = 3.14; // неявне перетворення 3.14 до типу int
```

Бібліотечні математичні функції

Загальні відомості. Використання та нюанси - лаб. заняття

cmath — бібліотека математичних функцій

#include <cmath>

Бібліотечні математичні функції

Загальні відомості. Використання та нюанси - лаб. заняття

cmath — бібліотека математичних функцій
#include <cmath>

sqrt, fabs, sin, cos, tan, asin, acos, atan, exp, log, pow, hypot

Бібліотечні математичні функції

Загальні відомості. Використання та нюанси - лаб. заняття

cmath — бібліотека математичних функцій
#include <cmath>

sqrt, fabs, sin, cos, tan, asin, acos, atan, exp, log, pow, hypot

isfinite, isnan, isinf
тощо

Бібліотечні математичні константи

Загальні відомості. Використання та нюанси - лаб. заняття

Використання констант M_PI, M_E (π, e відповідно) тощо потребує рядка, який має передувати включенню **cmath**.

```
#define _USE_MATH_DEFINES

#include <iostream>
      // якщо не потрібний, то включати не обов'язково

#include <cmath>
...
```

Якщо наявні інші включення, наприклад `#include <iostream>`, то вони можуть також використовувати бібліотеку **cmath**, тому зазначений рядок бажано записувати на самому початку.

Бібліотечні математичні константи

Загальні відомості. Використання та нюанси - лаб. заняття

Використання констант M_PI, M_E (π, e відповідно) тощо потребує рядка, який має передувати включенню **cmath**.

```
#define _USE_MATH_DEFINES

#include <iostream>
      // якщо не потрібний, то включати не обов'язково

#include <cmath>
...
```

Якщо наявні інші включення, наприклад `#include <iostream>`, то вони можуть також використовувати бібліотеку **cmath**, тому зазначений рядок бажано записувати на самому початку.

PS Ці імена не специфікуються стандартом, але підтримуються компіляторами.

Бібліотечні математичні константи - 2

Загальні відомості. Використання та нюанси - лаб. заняття

Бібліотека **numbers** містить означення деяких математичних констант.

```
#include <numbers>

...
numbers::pi + numbers::e
...
```

Ім'я **numbers** означене в просторі імен **std**, тому або `std::numbers::pi`, або слід задати

```
using namespace std;
```

Бібліотечні математичні функції та константи

Загальні відомості. Використання та нюанси - лаб. заняття

Чи є функція **tan** усюди визначеною? А в програмуванні?

Бібліотечні математичні функції та константи

Загальні відомості. Використання та нюанси - лаб. заняття

Чи є функція **tan** усюди визначеною? А в програмуванні?

Що буде, якщо обчислити вирази $tg\frac{\pi}{2}$, $tg\,\pi$? Як це пояснити?

Програмування
Лекція 4. Функції

Підпрограми

Задачі, програми, підзадачі, підпрограми

Задачу можна розкласти на частини — підзадачі.

Програма розв'язує задачу, підпрограма розв'язує підзадачу.

Виконання підпрограми задається її викликом, в якому вказано вирази, значення яких мають оброблятися під час її виконання.

Щоб отримати розв'язання задачі, у програмі використовують виклики підпрограм.

Підпрограми

Переваги використання підпрограм - 1

1. Підпрограми дозволяють краще структурувати код, зробити його більш зрозумілим.

За вдало названих підпрограм загальна програма, записана переважно викликами підпрограм, буде лаконічно повідомляти виконувані дії і читатися майже як розмовна мова, бо зайві деталі будуть приховані.

Дістатись з факультету в червоний корпус:

вийти з корпусу()
дійти до метро()
зайти в метро()
проїхати(7 зупинок у напрямку до центру)
вийти в метро()
дійти до червоного корпусу()

Те саме, записане деталізовано, може виявитись непридатним для швидкого розуміння людиною.

Підпрограми

Переваги використання підпрограм - 2

2. Підпрограми дозволяють звести розв'язання складної задачі до розв'язання низки простих та підвищити шанси задачі бути розв'язаною.

"Розділяй та пануй": велика задача розбивається на менші, менші розв'язуються окремо і далі з їх розв'язань збирається розв'язання великої задачі.

$\Rightarrow$ шанси розв'язати велику задачу суттєво зростають

$\Rightarrow$ можливість вести сумісне розроблення

Підпрограми

Переваги використання підпрограм - 3

3. Підпрограми дозволяють уникати дублювання коду і замість дублювання повторно використовувати один той самий код.

Дублювання коду виникає тоді, коли фрагменти коду повторюються. Може бути явним (текст збігається посимвольно) або прихованим (текст начебто різний, але задає по суті ті самі дії).

Підпрограми

Декомпозиція та абстракція — основи програмування з підпрограмами

Декомпозиція розкладає задачу на підзадачі.
Кожна підзадача розв'язується окремо.
Далі результат збирається до купи.

Абстракція: для збирання важливі не деталі реалізації, а зовнішня поведінка розв'язків підзадач, що задає їх спосіб використання.
Під час збирання підзадачі є чорними скриньками, що взаємодіють за чітким та заздалегідь визначеним протоколом.

Абстрагуємось від деталей реалізації. Важливі тільки інтерфейси використовуваних частин.

$\Rightarrow$ Результат збирається з блоків, які можуть розроблятись окремо.

$\Rightarrow$ Розроблення пришвидшується, можливе сумісне розроблення.

Підпрограми

Бібліотеки

Деякі підзадачі, наприклад:
— введення
— виведення
— обчислення математичних функцій
зустрічаються в різних задачах і є доволі стандартними.

Для стандартних підзадач у мовах програмування існують набори готових підпрограм, зібрані в бібліотеки.

Бібліотека — набір підпрограм (спрощений погляд).

Стандартна, або системна, бібліотека мови програмування — бібліотечні засоби, що входять у стандартну поставку транслятора.

Стандартна бібліотека додаткового встановлення не потребує.

Підпрограми

Бібліотеки

Стандартні підпрограми вміють багато.

Треба знати або їх, або хоча б про їх існування (і тоді за потреби відшукати в документації спосіб їх використання).

Чому? Використання готових деталей набагато легше, ніж побудова з “нуля”.

Абсолютно все передбачити неможливо. Тому програмісти змушені писати власні підпрограми.

За бажанням програміст може використовувати інші бібліотеки — сторонні бібліотеки, або third-party libraries.

Також програміст може розробляти власні бібліотеки.

У мові C++ усі підпрограми є функціями.

Погляд згори на роботу функцій

Функція та її виклик

Функція (у мовах програмування) — частина програми, оформлена в спеціальний спосіб.

Може повертати результат, а також змінювати значення, що зберігаються в пам'яті програми.

Застосування функції — виклик функції, є виразом

<ім'я функції> (перелік виразів, що задають оброблювані значення = аргументи виклику)

Уточнення: ім'я функції насправді є частковим випадком виразу, що задає функцію.

аргументи виклику = фактичні параметри

Погляд згори на роботу функцій

Виконання виклику функції

$x = \sin(1.0/7);$

— обчислюється вираз, що задає функцію
— обчислюються аргументи виклику (у прикладі він один)
— обчислені значення передаються функції
— керування передається функції
— функція виконує обчислення
— обчислення завершено: виклик функції завершується та функція повертає результат
— керування порядком обчислень повертається в точку, з якої було здійснено виклик
— у прикладі: продовжується виконання присвоювання

Написання та використання власних функцій

Означаємо власну функцію

```
double first(double x, double y) {
    double tmp;
    tmp = 15.0 * x + 9.0 * y;
    return tmp;
}
```

Складові:

заголовок

- тип результату
- ім'я
- перелік формальних параметрів

тіло (є блоком)

Блок — послідовність інструкцій у фігурних дужках.

Інструкція **return** задає завершення виконання виклику та повернення значення записаного в ній виразу як результату виклику функції.

Це приклад означення функції.

Написання та використання власних функцій

Приклади використання функцій у програмі

```
double res, x = 1.0, y = 2.0;
res = first(0.0, 0.0);            // 0.0
res = first(4.0 + 1.0, -2.0);     // 57.0
res = first(1, 0.0);              // 15.0
res = first(x, y);                // 33.0
res = first(y, x);                // 39.0
```

За кількістю перелік аргументів виклику має відповідати переліку формальних параметрів.

Типи аргументів виклику можуть бути типу **double** або сумісні з ним.

Тип результату фіксується в означенні функції і є **double**.

Написання та використання власних функцій

Приклади використання функцій у програмі

```
double second(double x, double y)
{
    return 15.0 * x + 9.0 * y;
}
```

Записали попередню функцію трохи більш компактно.

Написання та використання власних функцій

Приклади використання функцій у програмі

```
double third(double x, double y)
{
     x *= 15.0;
     y *= 9.0;
     return x + y;
}
. . . .
double res, x = 1.0, y = 2.0;
res = third(x, y);
cout << res << " " << x << " " << y << endl; // 33 1 2
```

Передача параметра за значенням (by value). У функції дії відбуваються над копіями значень.

Завершення виконання виклику та результат

Завершення виконання виклику функції

return <вираз>**;**

- Обчислюється вираз,
- отримане значення стає результатом, що повертається з виклику функції,
- виконання виклику завершується,
- продовжується виконуватись код, з якого був виклик

Якщо тип виразу відрізняється від типу результату функції, але сумісний з ним, то додатково виконується неявне перетворення типу.

Завершення виконання виклику та результат

Функція, що нічого не повертає

return;

```
void fourth(double x) {
     cout << x << "(double)" << endl;
     return;
     cout << "Це не буде виведено";
}
```

Типом результату маємо вказати **void**

Виклик записується у вигляді інструкції виразу:

```
fourth(13.13);
```

Завершення виконання виклику та результат

Завершення виконання виклику функції

```
void fifth()
{
     cout << "fifth";
}
```

Коли буде виконана остання інструкція тіла функції, то виконання виклику функції також завершиться і продовжить виконуватись код, з якого здійснювався виклик.

Функція, що змінює аргументи виклику

Щоб функція могла змінити змінні, що є аргументами виклику, необхідно повідомити їй, де вони розташовані в пам'яті.

Можна використати вказівники (буде пізніше) або посилання (reference).

Посилання — альтернативне ім'я змінної.

T& — тип посилань на дані типу **T**

Синтаксично змінна-посилання типу **T&** веде себе як змінна типу **T** (принаймні у тих контекстах, що розглядаємо зараз).

Функція, що змінює аргументи виклику

Посилання

```
int n1 = 0, n2 = 4;          |          n1    |          n2
int &m1 = n1, &m2 = n2;      |  m1   0        |  m2   4
m1 = 3;                      |       3        |       4
m2 = m1;                     |       3        |       3
m2 = 5;                      |       3        |       5
```

Функція, що змінює аргументи виклику

Особливості посилань

- завжди явно ініціалізуються
- мають ініціалізуватися l-виразами
- посилання змінити не можна (не можна змусити посилатися на іншу ділянку пам'яті), проте можна змінити вміст ділянки пам'яті, на яку воно посилається
- посилання на посилання заборонені
- посилання утворює "синонім" імені змінної
- ім'я змінної-посилання також утворює l-вираз

Функція, що змінює аргументи виклику

```
int sixth(int &n)
{
     n += 2;
     return n;
}
      . . .
     int m = 10;
     sixth(m);
     cout << m << endl;        // виведе 12
      . . .
```

- У тілі функції параметри-посилання позначають ділянки пам'яті змінних, які є аргументами її виклику.
- На місце параметра-посилання має підставлятися l-вираз.

Синтаксичні помилки: `sixth(3.5); sixth(m+1);`

Область дії оголошення, або Scope, імені

scope — область видимості, область дії оголошення

Є одним з механізмів, що дають можливість для різних сутностей програми використовувати однакові імена.

Область дії оголошення імені (name scope) — сукупність місць у програмі, де діє відповідне оголошення.

... — певний регіон коду, за винятком його частин, в яких діє інше оголошення того самого імені.

У моменти, коли оголошення змінної перекрито іншим оголошенням, змінна не знищується і продовжує існувати, хоча доступ до її значення є тимчасово втраченим.

Область дії оголошення імені: приклад

```
double third(double x, double y)
{
     x *= 15.0;
     y *= 9.0;
     return x + y;
}
...

double res, x = 1.0, y = 2.0;
third(x+1, y);
...
```

x, y з функції та нижче — “однофамільці”

Приклад

```
int a, b;
int f(int c, int &d) {
    int a = 3 + c;
    {   int c = 10;
        a = a + c + b;
    }
    d = c;
    c = 13;
    return a;
}
int main() {
    int a, b = 1, c = 3;
    a = f(b, c);
    cout << a << " " << c << endl;
    return 0;
}
```

код прикладу в sample.cpp

Локальні та глобальні імена

Локальними є імена:

- оголошені в блоках
- імена формальних параметрів функції

Відповідні змінні називають локальними.

Імена, що оголошені ззовні блоків, є глобальними. Відповідні змінні є глобальними (ЗЛО).

Оголошення, записане всередині блоку, діє до кінця блоку, за винятком частин, де перекривається іншим, вкладеним оголошенням.

Зовнішнє оголошення діє до кінця файлу, за винятком частин, де перекривається.

В одному блоці не можна оголосити дві змінні з одним тим самим іменем.

Класи пам'яті, або Storage duration

Класи пам'яті визначають спосіб керування часом існування змінних.

Глобальні змінні створюються на початку виконання програми і існують постійно.

Статичні змінні існують протягом виконання програми, завжди ініціалізуються.

Глобальні змінні є статичними, але не всі статичні є глобальними. Випадок thread_local не розглядаємо!

Автоматичні змінні — створюються за інструкцією, записаною в блоці, автоматично знищуються при виході з блоку.

Класи пам'яті, або Storage duration

Автоматичні змінні

Змінні, що відповідають формальним параметрам функції, гарантовано існують до кінця виконання тіла функції (можливо, трохи довше).

Локальні змінні функції, означені в її тілі, існують до кінця блоку, в якому були означені, і є **автоматичними** (якщо в явному вигляді не було сказано інше).

Автоматичні змінні функцій **не зберігають значення між викликами**.

Локальні змінні функції можуть бути не тільки автоматичними, але це має вказуватися в коді явно (поки не розглядаємо).

Класи пам'яті змінних

```
int a, b;                    // статичні
int f(int c, int &d) {
    int a = 3 + c;           // автоматична
    {   int c = 10;          // автоматична
        a = a + c + b;
    }
    d = c;
    c = 13;
    return a;
}
int main() {
    int a, b = 1, c = 3;  // автоматичні
    a = f(b, c);
    cout << a << " " << c << endl;
    return 0;
}
```

Прототип функції

Означення та оголошення

Означення: означає певну сутність (створює відповідний елемент у пам'яті програми)

```
double f(double x) {
     return x + 2;
}
int n;
```

Оголошення: повідомляє компілятору про певну сутність.

Оголошення функції – прототип функції. Складається з заголовку функції та ;

```
double f(double x);
```

або

```
double f(double);
```

- Деякі означення одночасно є оголошеннями.

Прототип функції

Одиниці трансляції

Файл з кодом (source file) $\rightarrow$
препроцесорні включення (#include) та виключення $\rightarrow$
одиниця трансляції (translation unit)
Одиниці трансляції окремо компілюються, далі збирається виконуваний файл.

• Кожне ім'я має бути оголошено до використання (у кожній одиниці трансляції, де використовується).

• Кожне використовуване ім'я повинно бути означене в одиниці трансляції не більше одного разу. (Означення, записані в блоках, до уваги не беремо. У блоці допускається перекривати означення зовнішнього імені.)
• За деякими винятками, кожна функція та змінна має бути означена в проєкті рівно один раз.

Допускаються як явні, так і неявні (= згенеровані компілятором) означення.

Прототип функції

Програма з однієї одиниці трансляції

Функції прийнято оголошувати на початку одиниці трансляції (зазвичай після включень), щоб забезпечити правило:

- Кожне ім'я має бути оголошено до використання.

Означення можуть розташовуватися після оголошень у довільному порядку (зазвичай, після main).

```
int f();
int main(){
    ...
    f();
    ...
}
int f(){
  return 13;
}
```

Прототип функції

Програма з кількох одиниць трансляції

Щоб використовувати в одній одиниці трансляції засоби, означені в іншій, їх треба оголосити.

Для кожної одиниці трансляції (за винятком тієї, де main), складають заголовний (header) файл, що містить оголошення (та коментар-документацію) означених у ній сутностей.

Традиція: Одиниця трансляції та відповідний їй заголовний файл мають відрізнятися виключно розширенням: наприклад, `unit.cpp` та `unit.h`.

За використання засобів, означених в `unit.cpp`, на початку інших файлів з текстом програми записують

```
#include "unit.h"
```

Включення файлів стандартної бібліотеки записуємо в кутових лапках (`<iostream>`), включення власних файлів записуємо в лапках (`"mylib.h"`).

Програмування

Лекція 5. Умови. Керування порядком обчислень

Оператори порівняння

Операції порівняння: порівняння чисел

оператори відношення (relational) >, <, >=, <=
оператори рівності (equality) ==, !=

Результат є значенням типу **bool** (значення **true**, **false**).

Арифметичні типи є сумісними в операціях порівняння. За потреби виконуються ті самі неявні стандартні арифметичні перетворення, що й у випадку арифметичних операцій.

Порівняння чисел відбувається за законами математики, єдине виключення: NaN≠NaN

Значення типу **bool** порівнюються як відповідні цілі значення:
для **true** це 1, для **false** — 0

Оператори порівняння

Операції порівняння: порівняння чисел: приклади

```
1 == 2          false
1 != 2          true

13 == 13.1      false
13 == 13.0      true

12 <= 5         false

-1.4 > true     false
false < true    true
13 > true       true

true == 1       true
true == 1.0     true
true == 2       false
```

Оператори порівняння

Операції порівняння: порівняння символів

До значень типу **char** застосовні усі оператори >, <, >=, <=, ==, !=.

Символи порівнюються за кодами.
(Перед порівнянням виконується перетворення на тип **int**, за знакового типу **char**)

Варто пам'ятати, що
0<...<9<A<B<...<Z<a<b<..<z

Як між собою порівнюється кирилиця та латиниця залежить від кодування і трохи від компілятора (тип char зазвичай є знаковим, але все-таки зазвичай).

Оператори порівняння

Операції порівняння: порівняння символів: приклади

```
'9' > '7'          true
'A' == 'a'         false
'f' < 'F'          false
'F' < 'f'          true

'c' == 'c'         false
```

У чому причина?

Насправді все чесно: одна латинська, інша кирилічна :)

```
'a' < 32          false
'c' == 'C'+' '    true
```

Логічні оператори

... у математиці застосовуються до логічних значень бінарні 'і', 'або' та унарна 'не'

A	B	A і B	A або B	не A
false	false	false	false	true
false	true	false	true	
true	false	false	true	false
true	true	true	true	

and або $\&\&$, **or** або $||$, **not** або **!**

Результатом обчислення завжди є значення типу **bool**

Логічні оператори

Оцінка істинності значення

Логічні оператори застосовні до значень довільних арифметичних типів, не тільки до значень типу **bool**.

Для цього необхідно вміти виконувати оцінку істинності значення (тобто має бути можливим неявне перетворення на тип **bool**).

Істинність нульових числових значень 0, 0.0, а також значень **false**, '\0' оцінюється як **false**.

Істинність ненульових числових значень, **true**, символів, код яких не нуль, оцінюється як **true**.

Логічні оператори

Оцінка істинності значення: приклади

```
bool('0')            true

bool(' ')            true

bool(0)              false

bool(-0.0)           false

bool(0.0000000001)   true
```

Логічні оператори

Обчислення **not**

not $expr$

обчислюється значення виразу $expr$ та оцінюється його істинність

якщо отримане значення оцінюється як **true**, то значенням виразу **not** $expr$ є **false**, інакше — **true**

```
not true     !true    false
not false    !false   true

not 0        !0       true
not -5.5     !-5.5    false
not 'a'      !'a'     false
not ' '      !' '     false
not '0'      !'0'     false
not '\0'     !'\0'    true
```

Логічні оператори

Обчислення **and** та **or**

Оператори **and** (&&) та **or** (||) обчислюються ледаче.

Аргументи обчислюються зліва направо. Як тільки результуюче значення стає зрозумілим, то обчислення завершуються.

expr1 **and** *expr2*
обчислюється значення виразу **bool**(*expr1*), якщо воно **false**, то стає значенням виразу і другий аргумент не обчислюється; інакше результатом обчислення стає значення виразу **bool**(*expr2*)

expr1 **or** *expr2*
обчислюється значення виразу **bool**(*expr1*), якщо воно **true**, то стає значенням виразу і другий аргумент не обчислюється; інакше результатом обчислення стає значення виразу **bool**(*expr2*)

Логічні оператори

Обчислення **and** (&&) та **or** (||): приклади

```
false and 13      false

0 and (0/0)       false

13 or 15.5        true
15.5 || 13        true

0 or 15           true
0 || 15           true

'a' and 13        true
'0' && '0'        true
```

Логічні оператори

Пріоритети операторів (за спаданням): оновлення

унарні $+$,$-$, !
$*$, $/$, $\%$ (мультиплікативні)
бінарні $+$,$-$ (адитивні)
відношення: <, <=, >, >=
рівності: !=, ==
логічне І: $\&\&$
логічне АБО: $||$

Стиль запису порівнянь: праворуч читається легше

```
a < b && b > c        (a < b) && (b > c)
```

Умови, або Умовні вирази

... — вирази, які можуть оцінюватись як істинні або хибні
... є основою прийняття рішень з вибору подальшого шляху обчислень
... значення умови може не бути типу **bool**, але має існувати механізм оцінювання істинності (неявного приведення до типу **bool**)

Часто умова є ознакою певної властивості.

Записати умову того, що рівняння $ax + b = 0$ має єдиний розв'язок.
Відповідь: a!=0.0 (умова a виражає те саме)

Для побудови умов найчастіше використовують порівняння, логічні оператори, а також функції перевірки властивостей, наприклад: $isfinite$, та не тільки.

Умови, або Умовні вирази

Приклад запису умови

Записати умову того, що $x \in [1.5, 3.5]$.

Відповідь: $(1.5 <= x)\&\&(x <= 3.5)$.

Як обчислюється вираз $1.5 <= x <= 3.5$?

$(1.5 <= x) <= 3.5$
І це зовсім не те, що мається на увазі в математиці.

Умови, або Умовні вирази

Приклад запису умови

Записати умову того, що числа x, y, z попарно рівні.

Відповідь: $(x == y)\&\&(y == z)$.

Як обчислюється вираз $x == y == z$?

$(x == y) == z$
І це зовсім не те, що треба в задачі.

Умови, або Умовні вирази

Приклад запису умови

Записати умову того, що ціле число a ділиться націло на 2025.

Відповідь: $a\%2025 == 0$, або $!(a\%2025)$

Умови, або Умовні вирази

Приклад запису умови

Записати умову того, що гарантовано цілі та ненульові числа a, b, c утворюють геометричну прогресію.

Математика питання: $\frac{b}{a} = \frac{c}{b}$

Але безпосередньо через ділення записувати не можна: за цілочисельного ділення можлива втрата точності, за дійсного ділення можлива похибка обчислень.

Відповідь: $b * b == a * c$

Умови, або Умовні вирази

Приклад запису умови

Записати умову того, що з відрізків з довжинами a, b, c можна утворити трикутник.

Математично: слід перевірити
довжини невід'ємні: $a \geq 0$, $b \geq 0$, $c \geq 0$,
нерівність трикутника: $a + b > c$, $a + c > b$, $b + c > a$

Записувати всі ці частини безпосередньо в одному виразі через **and** не слід, бо тоді отримаємо довгу умову, яка є складною для сприйняття та перевірки.

Довгих та складних умов слід уникати, розбиваючи їх на більш прості складові.

Оформимо частини умови допоміжними функціями.

Умови, або Умовні вирази

Приклад запису умови - нерівність трикутника

Умову того, що виконується нерівність трикутника, оформимо окремою функцією

```
bool triangle_inequality(double a, double b, double c)
// Check triangle inequality.
{
     return (a + b > c) and
            (a + c > b) and (b + c > a);
}
```

Умови, або Умовні вирази

Приклад запису умови - невід'ємність довжин

Умову невід'ємності значень записувати як функцію від трьох аргументів, тобто

```
bool is_lengths(double a, double b, double c) { // Hi!
    return (a >= 0) and (b >= 0) and (c >= 0);
}
```

дуже неправильно! бо ця функція безпосередньо розв'язує кілька незалежних задач одночасно

Має бути:

```
bool is_length(double x)
{
     return (x >= 0);
}
```

Умови, або Умовні вирази

Приклад запису умови - остаточний вигляд

Записати умову того, що з відрізків з довжинами a, b, c можна утворити трикутник.

```
is_length(a) and is_length(b) and is_length(c)
   and triangle_inequality(a, b, c)
```

Інструкція розгалуження, або Умовна інструкція

... інструкція, що задає вибір одного з декількох шляхів обчислень

Загальний вигляд:
if (умова) інструкція
або
if (умова) інструкція1 **else** інструкція2

Інструкція розгалуження, або Умовна інструкція

Приклади

```
int n = 13;
if (n%2 == 0)
    cout<<"even";
else
    cout<<"odd";
```

```
int n = 13;
if (!(n%2))
    cout<<"even";
else
    cout<<"odd";
```

буде виведено

odd

Інструкція розгалуження, або Умовна інструкція

Приклади

```
bool b = true;
if (b)
     cout << "TRUE";
else
     cout << "FALSE";
```

буде виведено

TRUE

Інструкція розгалуження, або Умовна інструкція

Приклади

```
int n = 12;
if (n)
  cout << "Число не нульове.";
```

буде виведено
Число не нульове.

Але краще навіть одну інструкцію записувати як блок (=послідовність інструкцій у фігурних дужках).

```
int n = 12;
if (n)
{
  cout << "Число не нульове.";
}
```

Інструкція розгалуження, або Умовна інструкція

Вкладені умовні інструкції

```
if (умова1)
    інструкція1
else if (умова2)
    інструкція2
...
else if (умоваN)
    інструкціяN
else
    інструкція
```

Якщо одна гілка спрацювала, то на цьому виконання інструкції завершується.

Інструкція розгалуження, або Умовна інструкція

Приклади

```
int n = 12;
if (n < 0)
  cout << "1";
else if (n < 100)
  cout << "2";
else
  cout << "3";
```

```
2
```

Інструкція розгалуження, або Умовна інструкція

Приклади

```
int n = 12;
if (n)
  cout << "1";
else if (n)
  cout << "2";
else
  cout << "3";
```

```
1
```

Інструкція розгалуження, або Умовна інструкція

Приклади

Обчислення $(-1)^n$.

```
int power_1_(int n)
{
     if (n % 2)
          return -1;
     else
          return 1;
}
```

Інструкція розгалуження, або Умовна інструкція

Приклад

Написати фрагмент коду, що обчислює вираз:

$$f(x) = \begin{cases} x, & \text{якщо } 3 \leq x \leq 7, \\ x+2, & \text{якщо } x > 7, \\ 2x, & \text{якщо } -5 < x < 3, \\ 0, & \text{якщо } x \leq -5, \end{cases}$$

Безпосередньо НЕ програмуємо. Спочатку впорядковуємо інтервали.

$$f(x) = \begin{cases} 0, & \text{якщо } x \leq -5, \\ 2x, & \text{якщо } -5 < x < 3, \\ x, & \text{якщо } 3 \leq x \leq 7, \\ x+2, & \text{якщо } x > 7, \end{cases}$$

Інструкція розгалуження, або Умовна інструкція

Приклад - продовження

$$f(x) = \begin{cases} 0, & \text{якщо } x \leq -5, \\ 2x, & \text{якщо } -5 < x < 3, \\ x, & \text{якщо } 3 \leq x \leq 7, \\ x+2, & \text{якщо } x > 7, \end{cases}$$

```
if (x <= -5)
  res = 0;
else if (x < 3)
  res = 2 * x;
else if (x <= 7)
  res = x;
else
  res = x + 2;
```

Інструкція розгалуження, або Умовна інструкція

Операція розгалуження

умовна операція, або операція розгалуження
тернарний оператор ?:

$condition ? expr1 : expr2$

$(-1)^n$	**n%2 ? -1 : 1**
$min(a, b)$	**a<=b ? a : b**

Примітка. У C++ є функція min.

Кілька слів про стиль та значення, що повертаються з функцій

Функції, що задають умови - 1

Пишемо так:

```
return x < y;
```

Так не пишемо:

```
if (x < y)
   return true;
else
   return false;
```

Так не пишемо:

```
if (x < y)
   return true;
return false;
```

Пишемо так:

return $expr$;

Так не пишемо:

if ($expr$)
   return true;
else
   return false;

Так не пишемо:

if ($expr$)
   return true;
return false;

Кілька слів про стиль та значення, що повертаються з функцій

Функції, що задають умови - 2

Пишемо так:

return **not** ($expr$);

Так не пишемо:

```
if ( not(expr))
   return true;
else
   return false;
```

Так не пишемо:

```
if (not(expr))
   return true;
return false;
```

Пишемо так:

return **not** ($expr$);

Так не пишемо:

```
if (expr)
   return false;
else
   return true;
```

Так не пишемо:

```
if (expr)
   return false;
return true;
```

Кілька слів про стиль та значення, що повертаються з функцій

Функції, що повертають значення - 1

Якщо функція має повернути значення, то кожна гілка обчислень має завершуватись інструкцією **return**, що явно повертає значення.

Обчислення зазвичай програмують за такою схемою: спочатку відсікають тривіальні та специфічні випадки, потім записують код основного випадку.

```
int f(int n, int m) {
  if (m == 0)
    return 0;
  // основний випадок:
  return n / m;
}
```

Кілька слів про стиль та значення, що повертаються з функцій

Функції, що повертають значення - 2

На цій функції виникає **невизначена поведінка**, бо не всі гілки обчислень явно повертають значення!

```
double f(double x)
{
if (x!=3) return 1/(x-3);
}
```

Інструкція switch

Інструкція вибору варіанту, або перемикач (switch)

switch(селектор) {
case Мітка1: послідовність інструкцій 1
case Мітка2: послідовність інструкцій 2
...
case МіткаN: послідовність інструкцій N
default: послідовність інструкцій
}

— обчислюється значення селектора;
— обчислене значення послідовно порівнюється із мітками;
— за співпадіння зі значенням мітки керування передається на інструкцію, що слідує за цією міткою і далі послідовно виконується решта інструкцій; на інструкції **break;** виконання перемикача завершується;
— якщо значення не співпало з жодною міткою і наявна мітка **default**, то керування передається на інструкцію після неї.

Інструкція switch

Інструкція вибору варіанту, або перемикач (switch)

```
bool ask_yes_no(){
 int answer;
 bool res;
 cout << "Enter 0 for No or 1 for Yes: ";
 cin >> answer;
 switch (answer){
 case 0:
   res = false; break;
 case 1:
   res = true; break;
 default:
   cout << "Incorrect input. Let it be No." << endl;
   res = false;
 }
 return res;
}
```

Програмування

Лекція 6. Оброблення помилок. Винятки

Оброблення помилкових та неочікуваних ситуацій

Обробка помилок

Обчислення виразу $0/0$ може стати причиною аварійного завершення програми. Якби цього уникнути...

Оброблення помилкових та неочікуваних ситуацій

Обробка помилок

Обчислення виразу $0/0$ може стати причиною аварійного завершення програми. Якби цього уникнути...

Існує дві базові стратегії оброблення помилок:

Оброблення помилкових та неочікуваних ситуацій

Обробка помилок

Обчислення виразу $0/0$ може стати причиною аварійного завершення програми. Якби цього уникнути...

Існує дві базові стратегії оброблення помилок:

— помилок можна уникати
виконувати попередні перевірки коректності значень аргументів

Оброблення помилкових та неочікуваних ситуацій

Обробка помилок

Обчислення виразу $0/0$ може стати причиною аварійного завершення програми. Якби цього уникнути...

Існує дві базові стратегії оброблення помилок:

— помилок можна уникати
виконувати попередні перевірки коректності значень аргументів

— можна ліквідовувати їх наслідки
обробляти відмови та винятки, що були згенеровані під час виконання
(але не у випадку цілочисельного ділення на нуль)

Оброблення помилкових та неочікуваних ситуацій

Поняття винятку

Виняткова ситуація, або виняток (exception) — ситуація, що не дозволяє продовжити звичайне виконання програми.

Наприклад: не те ввели, намагались поділити на нуль.

Оброблення помилкових та неочікуваних ситуацій

Поняття винятку

Виняткова ситуація, або виняток (exception) — ситуація, що не дозволяє продовжити звичайне виконання програми.

Наприклад: не те ввели, намагались поділити на нуль.

Виникнення виняткової ситуації вимагає спеціальних дій з її розв'язання, тобто вимагає оброблення помилок.

Оброблення помилкових та неочікуваних ситуацій

Поняття винятку

Виняткова ситуація, або виняток (exception) — ситуація, що не дозволяє продовжити звичайне виконання програми.

Наприклад: не те ввели, намагались поділити на нуль.

Виникнення виняткової ситуації вимагає спеціальних дій з її розв'язання, тобто вимагає оброблення помилок.

Щоб відокремити оброблення помилок від основної логіки програми, використовують механізм винятків.

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Синхронні винятки генеруються виконуваним кодом (бібліотеками, власним кодом).

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Синхронні винятки генеруються виконуваним кодом (бібліотеками, власним кодом).

Асинхронні винятки надходять ззовні. Найчастіше, апаратні. Наприклад, сигнали процесора.

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Синхронні винятки генеруються виконуваним кодом (бібліотеками, власним кодом).

Асинхронні винятки надходять ззовні. Найчастіше, апаратні. Наприклад, сигнали процесора.

За появи винятку, якщо цей виняток не обробити, то роботу програми буде аварійно зупинено.

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Синхронні винятки генеруються виконуваним кодом (бібліотеками, власним кодом).

Асинхронні винятки надходять ззовні. Найчастіше, апаратні. Наприклад, сигнали процесора.

За появи винятку, якщо цей виняток не обробити, то роботу програми буде аварійно зупинено.

У мові C++ за стандартом винятки синхронні.
Деякі компілятори вміють генерувати код, що здатен обробляти асинхронні винятки.

Оброблення помилкових та неочікуваних ситуацій

Як виникають винятки

Винятки бувають синхронні та асинхронні.

Синхронні винятки генеруються виконуваним кодом (бібліотеками, власним кодом).

Асинхронні винятки надходять ззовні. Найчастіше, апаратні. Наприклад, сигнали процесора.

За появи винятку, якщо цей виняток не обробити, то роботу програми буде аварійно зупинено.

У мові C++ за стандартом винятки синхронні.
Деякі компілятори вміють генерувати код, що здатен обробляти асинхронні винятки.

Увага! Для компілятора MSVC треба вмикати опцію /EHsc, щоб він працював за стандартом. (Можна вибрати відповідну опцію на вкладці Генерація коду у властивостях проєкту.)

Оброблення помилкових та неочікуваних ситуацій

Виникнення та оброблення виняткових ситуацій

Програмні засоби (бібліотечні засоби) або власний код повідомляють про виникнення виняткової ситуації: кидають, або генерують, виняток.

Оброблення помилкових та неочікуваних ситуацій

Виникнення та оброблення виняткових ситуацій

Програмні засоби (бібліотечні засоби) або власний код повідомляють про виникнення виняткової ситуації: кидають, або генерують, виняток.

Під винятком одночасно розуміють і саму ситуацію, і значення, що містить інформацію про неї.

to raise an exception, to throw an exception

Оброблення помилкових та неочікуваних ситуацій

Виникнення та оброблення виняткових ситуацій

Програмні засоби (бібліотечні засоби) або власний код повідомляють про виникнення виняткової ситуації: кидають, або генерують, виняток.

Під винятком одночасно розуміють і саму ситуацію, і значення, що містить інформацію про неї.

to raise an exception, to throw an exception

В іншому місці програми має бути код, що знає, що робити в цій ситуації. Виконання цього коду — оброблення винятку.

обробник, handler, обробити виняток, to handle an exception
пастка, catch, зловити виняток, to catch an exception

Оброблення помилкових та неочікуваних ситуацій

Переваги використання винятків

Правило 80/20

20% коду забезпечують 80% функціональності програми;
80% коду обробляє нетипові ситуації та помилки.

Оброблення помилкових та неочікуваних ситуацій

Переваги використання винятків

Правило 80/20

20% коду забезпечують 80% функціональності програми;
80% коду обробляє нетипові ситуації та помилки.

• Винятки дозволяють локалізувати оброблення помилок в одній (чи майже в одній) точці коду, не розподіляючи його по всьому коду.

Оброблення помилкових та неочікуваних ситуацій

Переваги використання винятків

Правило 80/20

20% коду забезпечують 80% функціональності програми;
80% коду обробляє нетипові ситуації та помилки.

• Винятки дозволяють локалізувати оброблення помилок в одній (чи майже в одній) точці коду, не розподіляючи його по всьому коду.

⇒ Код оброблення помилок відокремлюється від основного коду.

Оброблення помилкових та неочікуваних ситуацій

Переваги використання винятків

Правило 80/20

20% коду забезпечують 80% функціональності програми;
80% коду обробляє нетипові ситуації та помилки.

• Винятки дозволяють локалізувати оброблення помилок в одній (чи майже в одній) точці коду, не розподіляючи його по всьому коду.

⇒ Код оброблення помилок відокремлюється від основного коду.

⇒ Суть коду стає більш зрозумілою, оскільки код не захаращений деталями оброблення помилок.

Оброблення помилкових та неочікуваних ситуацій

Недоліки використання винятків

- За використання винятків порушується структурованість коду: відслідкувати точку, в яку передається керування, може бути доволі складно.

Оброблення помилкових та неочікуваних ситуацій

Недоліки використання винятків

- За використання винятків порушується структурованість коду: відслідкувати точку, в яку передається керування, може бути доволі складно.
- Реалізація механізму винятків не пришвидшує виконання коду.

Оброблення помилкових та неочікуваних ситуацій

Недоліки використання винятків

- За використання винятків порушується структурованість коду: відслідкувати точку, в яку передається керування, може бути доволі складно.
- Реалізація механізму винятків не пришвидшує виконання коду.

Щодо першої проблеми, то якщо все оброблення помилок локалізоване, то знайти потрібне місце не буде проблемою.

Оброблення помилкових та неочікуваних ситуацій

Недоліки використання винятків

- За використання винятків порушується структурованість коду: відслідкувати точку, в яку передається керування, може бути доволі складно.
- Реалізація механізму винятків не пришвидшує виконання коду.

Щодо першої проблеми, то якщо все оброблення помилок локалізоване, то знайти потрібне місце не буде проблемою.

Щодо другої, то сучасні компілятори мають можливість генерувати код, що не підтримує оброблення винятків. Тому, якщо швидкодія дуже принципова, то обійтись без винятків можливо.

Механізм винятків

Приклад програми з обробленням винятків

`exceptions.cpp`

Механізм винятків

Інструкція **try**

```
try {
    double x = 0., y = 0.;
    x = inputDouble();
    y = mySqrt(x);
    cout << "for x=" << x << " y=" << y << endl;
} catch (logic_error &e) {
    cout << e.what() << endl;
} catch (...) {
    cout << "smth went wrong" << endl;
}
```

Механізм винятків

Інструкція **try**: код, що охороняється

```
try block0
catch (тип1 var) block1
catch (тип2 var) block2
...
catch (типN var) blockN
catch (...) block(N+1)
```

`block` – блок, або складена інструкція, – послідовність інструкцій у фігурних дужках, н-д: {int n; n=3;}

Механізм винятків

Інструкція **try**: код, що охороняється

```
try block0
catch (тип1 var) block1
catch (тип2 var) block2
...
catch (типN var) blockN
catch (...) block(N+1)
```

`block` – блок, або складена інструкція, – послідовність інструкцій у фігурних дужках, н-д: {int n; n=3;}

Блок `block0` є кодом, що охороняється (guarded section of code).

Під час його виконання перевіряється поява винятків. Якщо виняток з'являється, він може бути перехоплений та оброблений.

Механізм винятків

Інструкція **try**: код, що охороняється

```
try block0
catch (тип1 var) block1
catch (тип2 var) block2
...
catch (типN var) blockN
catch (...) block(N+1)
```

`block` – блок, або складена інструкція, – послідовність інструкцій у фігурних дужках, н-д: {int n; n=3;}

Блок `block0` є кодом, що охороняється (guarded section of code).

Під час його виконання перевіряється поява винятків. Якщо виняток з'являється, він може бути перехоплений та оброблений.

Маємо передбачити наявність обробника.

Механізм винятків

Що охороняє інструкція **try**

```
try block0
catch (  ) { }
catch (  ) { }
...
catch (  ) { }
catch (...) { }
```

Інструкція **try** охороняє тільки код блоку **block0**.

Механізм винятків

Що охороняє інструкція **try**

```
try block0
catch (  ) { }
catch (  ) { }
...
catch (  ) { }
catch (...) { }
```

Інструкція **try** охороняє тільки код блоку **block0**.

Винятки, що були згенеровані під час виконання її пасток, вона перехопити та обробити не може. Вони мають оброблятися пастками деякої **іншої** інструкції **try**, під час виконання якої розпочалось виконання цієї **try**.

Механізм винятків

Що охороняє інструкція **try**

```
try // (1)
  {цей код охороняється інструкцією (1)}
catch (тип)
  {цей код не охороняється інструкцією (1)}
catch (...)
  {цей код не охороняється інструкцією (1)}
```

Механізм винятків

Що охороняє інструкція **try**

```
try // (2)
  {код цього блоку охороняється інструкцією (2)
  ...
  try // (1)
    {цей код охороняється інструкціями (1) та (2)}
  catch (тип)
    {цей код не охороняється інструкцією (1),
     але він охороняється інструкцією (2)}
  catch (...)
    {цей код не охороняється інструкцією (1),
    але він охороняється інструкцією (2)}
  ...
  }
catch (...)
  {цей код не охороняється інструкцією (2)}
```

Механізм винятків

Інструкція **try**: пастка для винятків

Пастка, або обробник винятку, може мати вигляд:

```
catch (тип) block
catch (тип var) block
catch (...) block
```

Механізм винятків

Інструкція **try**: пастка для винятків

Пастка, або обробник винятку, може мати вигляд:

```
catch (тип) block
catch (тип var) block
catch (...) block
```

Записані в ній інструкції обробляють виняток.

Механізм винятків

Інструкція **try**: пастка для винятків

Пастка, або обробник винятку, може мати вигляд:

```
catch (тип) block
catch (тип var) block
catch (...) block
```

Записані в ній інструкції обробляють виняток.

Пастка призначена обробляти винятки певного (з точністю до & та const) типу (та похідні від нього).

Механізм винятків

Інструкція **try**: пастка для винятків

Пастка, або обробник винятку, може мати вигляд:

```
catch (тип) block
catch (тип var) block
catch (...) block
```

Записані в ній інструкції обробляють виняток.

Пастка призначена обробляти винятки певного (з точністю до & та const) типу (та похідні від нього).

За необхідності доступу до значення винятку вказують ім'я змінної, яка в пастці буде позначати це значення. Ця змінна буде існувати тільки впродовж виконання коду пастки.

Механізм винятків

Інструкція **try**: пастка для винятків

Пастка, або обробник винятку, може мати вигляд:

```
catch (тип) block
catch (тип var) block
catch (...) block
```

Записані в ній інструкції обробляють виняток.

Пастка призначена обробляти винятки певного (з точністю до & та const) типу (та похідні від нього).

За необхідності доступу до значення винятку вказують ім'я змінної, яка в пастці буде позначати це значення. Ця змінна буде існувати тільки впродовж виконання коду пастки.

Пастка `catch (...)` ловить довільні винятки і, якщо наявна, має бути останньою.

Механізм винятків –

```
try { // повна версія у exceptions.cpp
    double x = 0.0, y = 0.0;
    x = inputDouble();
    y = mySqrt(x);
    cout << "for x=" << x << " y=" << y << endl;
} catch (logic_error &e) {
    cout << e.what() << endl;
} catch (exception &e) {
    cout << e.what() << "... not expected" << endl;
} catch (...) {
    cout << "smth went wrong" << endl;
}
cout << "The End." << endl;
```

Що виконується на момент виникнення проблеми (введено -1): try-інструкція (має пастку для нашого винятку), її try-блок, виклик mySqrt.

Механізм винятків –

```
try { // повна версія у exceptions.cpp
    double x = 0.0, y = 0.0;
    x = inputDouble();
    y = mySqrt(x);
    cout << "for x=" << x << " y=" << y << endl;
} catch (logic_error &e) {
    cout << e.what() << endl;
} catch (exception &e) {
    cout << e.what() << "... not expected" << endl;
} catch (...) {
    cout << "smth went wrong" << endl;
}
cout << "The End." << endl;
```

Що виконується після знаходження обробника:
try-інструкція (має пастку для нашого винятку), її catch-блок для типу logic_error&

Механізм винятків –

```
try { // повна версія у exceptions.cpp
    double x = 0.0, y = 0.0;
    x = inputDouble();
    y = mySqrt(x);
    cout << "for x=" << x << " y=" << y << endl;
} catch (logic_error &e) {
    cout << e.what() << endl;
} catch (exception &e) {
    cout << e.what() << "... not expected" << endl;
} catch (...) {
    cout << "smth went wrong" << endl;
}
cout << "The End." << endl; // (*)
```

Що буде виконуватись після виконання обробника:
інструкція (*), наступна за try-інструкцією.

Механізм винятків

Пошук обробника винятку: коротке резюме

За появи винятку активується процес пошуку обробника.

Stack unwinding (розмотування стеку): Послідовно припиняється виконання всіх інструкцій та викликів, поки не буде припинено виконання **try**-блоку інструкції **try**, для якої існує пастка, що може обробити виняток цього типу.

Керування передається знайденій пастці.
Виняток вважається обробленим, а пастка активованою.
Виконується код пастки.

Якщо б були інші пастки, то вони не перевірялись би.

(У прикладі далі виконується інструкція, наступна за інструкцією try.)

Якщо обробник не буде знайдений, то програма аварійно завершиться.

Механізм винятків

Стек

Інструкції та виклики функцій можуть бути вкладеними

```
if (...) {
  if (...){
    f();
  }
}
```

Механізм винятків

Стек

Інструкції та виклики функцій можуть бути вкладеними

```
if (...) {
  if (...){
    f();
  }
}
```

Послідовність розпочатих, але незавершених інструкцій та викликів, запам'ятовується.
Ця допоміжна структура для керування обчисленнями живе за принципом LIFO (Last In First Out): виконання розпочатої останньою інструкції/виклику завершується першим.

Структуру, що живе за принципом LIFO, прийнято називати стеком.

Пам'ять процесу виконання програми

- код
- статичні змінні та значення рядкових констант
- програмний стек (автоматичні змінні, дані викликів)
- інше

Вміст програмного стеку, на прикладі виклику `demo_program_stack()`

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Від **exception** походять, зокрема, такі бібліотечні типи винятків:
logic_error, **runtime_error**

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Від **exception** походять, зокрема, такі бібліотечні типи винятків:
logic_error, **runtime_error**

Похідні від **logic_error**:
domain_error, **invalid_argument**, **length_error**, **out_of_range**

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Від **exception** походять, зокрема, такі бібліотечні типи винятків:
logic_error, **runtime_error**

Похідні від **logic_error**:
domain_error, **invalid_argument**, **length_error**, **out_of_range**

Похідні від **runtime_error**:
system_error, **range_error**, **overflow_error**, **underflow_error**

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Від **exception** походять, зокрема, такі бібліотечні типи винятків:
logic_error, **runtime_error**

Похідні від **logic_error**:
domain_error, **invalid_argument**, **length_error**, **out_of_range**

Похідні від **runtime_error**:
system_error, **range_error**, **overflow_error**, **underflow_error**

Похідні від **system_error**: **ios_base::failure**.

Класи винятків

Виняток може бути довільного типу. Але краще, щоб його тип був похідним від бібліотечного типу **exception**.

Від **exception** походять, зокрема, такі бібліотечні типи винятків:
logic_error, **runtime_error**

Похідні від **logic_error**:
domain_error, **invalid_argument**, **length_error**, **out_of_range**

Похідні від **runtime_error**:
system_error, **range_error**, **overflow_error**, **underflow_error**

Похідні від **system_error**: **ios_base::failure**.

Можна створювати власні типи винятків (2й семестр). Краще щоб вони були нащадками від **exception**. Краще не безпосередніми.

Порядок пасток

Інструкція **try** може мати кілька пасток.

Порядок пасток

Інструкція **try** може мати кілька пасток.

Якщо виняток може бути оброблений кількома пастками, то вибирається та, що записана першою.
Після її активації інші пастки активовані вже не будуть.

Порядок пасток

Інструкція **try** може мати кілька пасток.

Якщо виняток може бути оброблений кількома пастками, то вибирається та, що записана першою.
Після її активації інші пастки активовані вже не будуть.

Звідси випливає порядок запису пасток:
— спочатку записуємо більш конкретні пастки, потім — більш загальні;
— якщо жоден з типів не є нащадком іншого, то порядок відповідних цим типам пасток не принциповий.

Порядок пасток

Інструкція **try** може мати кілька пасток.

Якщо виняток може бути оброблений кількома пастками, то вибирається та, що записана першою.
Після її активації інші пастки активовані вже не будуть.

Звідси випливає порядок запису пасток:
— спочатку записуємо більш конкретні пастки, потім — більш загальні;
— якщо жоден з типів не є нащадком іншого, то порядок відповідних цим типам пасток не принциповий.

try {...}
catch (exception&) {...}
catch (logic_error&) {...}
Помилка! Пастка для **logic_error** ніколи активована не буде!

Порядок пасток

Інструкція **try** може мати кілька пасток.

Якщо виняток може бути оброблений кількома пастками, то вибирається та, що записана першою.
Після її активації інші пастки активовані вже не будуть.

Звідси випливає порядок запису пасток:
— спочатку записуємо більш конкретні пастки, потім — більш загальні;
— якщо жоден з типів не є нащадком іншого, то порядок відповідних цим типам пасток не принциповий.

Має бути так:
try {...}
catch (logic_error&) {...}
catch (exception&) {...}

Порядок пасток

Інструкція **try** може мати кілька пасток.

Якщо виняток може бути оброблений кількома пастками, то вибирається та, що записана першою.
Після її активації інші пастки активовані вже не будуть.

Звідси випливає порядок запису пасток:
— спочатку записуємо більш конкретні пастки, потім — більш загальні;
— якщо жоден з типів не є нащадком іншого, то порядок відповідних цим типам пасток не принциповий.

logic_error, **runtime_error** — між собою в довільному порядку
catch (exception&){...}
catch(...){...}

Механізм винятків

Розмір блоку, що охороняється

Дуже малий $\Rightarrow$

- можна зробити дуже деталізовані локалізацію та оброблення помилок
- код буде захаращений кодом оброблення помилок
- код буде малозрозумілий та повільний
- перевага винятків буде втрачена

Механізм винятків

Розмір блоку, що охороняється

Дуже малий $\Rightarrow$

- можна зробити дуже деталізовані локалізацію та оброблення помилок
- код буде захаращений кодом оброблення помилок
- код буде малозрозумілий та повільний
- перевага винятків буде втрачена

Дуже великий $\Rightarrow$

- причина появи винятку може не бути однозначно зрозумілою
- оброблення може звестись до завершення (не аварійного) програми

Механізм винятків

Розмір блоку, що охороняється: що робити?

У багатьох випадках власна ієрархія класів винятків допоможе знайти компроміс між деталізацією проблем та розмірами **try**-блоків та зібрати левову частку оброблення помилок до купи в кількох місцях на доволі верхньому рівні коду.

Механізм винятків

Розмір блоку, що охороняється: що робити?

У багатьох випадках власна ієрархія класів винятків допоможе знайти компроміс між деталізацією проблем та розмірами **try**-блоків та зібрати левову частку оброблення помилок до купи в кількох місцях на доволі верхньому рівні коду.

Завжди варто враховувати вимоги до оброблення помилок та здоровий глузд і не подрібнювати **try**-інструкції там, де нема потреби це робити.

Механізм винятків

Більш точна діагностика помилок

Оброблення винятків можна розподілити по рівням програми.

Механізм винятків

Більш точна діагностика помилок

Оброблення винятків можна розподілити по рівням програми.

На найвищому рівні обробляють системні та критичні помилки. Більш детальні помилки можна ідентифіковувати на нижніх рівнях, а далі через механізм винятків передавати оброблення на більш високий рівень програми (можливо, з частковим обробленням на місці).

• Гарна практика: за використання механізму винятків на верхньому рівні завжди записувати пастку для всіх винятків (щоб перехопити все те, що не було перехоплено до того).

Механізм винятків

Оператор **throw**

Так, саме оператор (operator), а не інструкція (statement). Хоча типом виразу є **void**.
... генерує (чи перекидує) виняток та розпочинає пошук обробника винятку.
throw
throw <вираз>

Вираз задає значення винятку. Найчастіше вираз має вигляд:
<тип винятків>(перелік аргументів)

Механізм винятків

Оператор **throw**

Так, саме оператор (operator), а не інструкція (statement). Хоча типом виразу є **void**.
... генерує (чи перекидує) виняток та розпочинає пошук обробника винятку.
throw
throw <вираз>

Вираз задає значення винятку. Найчастіше вираз має вигляд:
<тип винятків>(перелік аргументів)

Зазвичай аргументи запам'ятовуються в значенні винятку.

Механізм винятків

Оператор **throw**

Так, саме оператор (operator), а не інструкція (statement). Хоча типом виразу є **void**.
... генерує (чи перекидує) виняток та розпочинає пошук обробника винятку.
throw
throw <вираз>

Вираз задає значення винятку. Найчастіше вираз має вигляд:
<тип винятків>(перелік аргументів)

Зазвичай аргументи запам'ятовуються в значенні винятку.

Які аргументи можна вказувати залежить від типу винятків.

Більшість бібліотечних (крім **exception**) можуть прийняти рядок, який потім буде доступний за виразом **e.what()**, де e – ім'я, що в активованій пастці позначає виняток.

Механізм винятків

Оператор **throw**: перекидування на гору того самого винятку

```
try {
  // ...
} catch (     ) {
  // частково обробляємо
  throw; // перекидуємо на гору іншому обробнику
}
```

Специфікація винятків

Для функцій можна вказувати **специфікацію винятків**.
Якщо не вказати, то вважається, що функція може генерувати винятки.
Функція генерує винятки = під час її виконання (або коду, який вона викликає) можуть виникати винятки і функція їх не обробляє.

```
int g0();
int g1() noexcept;
int g2() noexcept(true);
int g3() noexcept(false);
```

Якщо **noexcept**-функція генерує виняток, то програма аварійно завершується.

Специфікація винятків

Приклади

```
void f1() // noexcept(false)
{
    throw logic_error("Smth wrong!");
}

void f2() // noexcept(false)
{
    f1();
}

void f3() noexcept
{
    try { f2(); }
    catch (logic_error&) {  }
}
```

Стратегії оброблення помилок та неочікуваних ситуацій

Що може зробити програма за наявності помилок, наприклад, у вхідних даних?

— аварійно завершитись;

— мовчки навернутись (прикинутись, що все гаразд, і завершити виконання без жодних повідомлень);

— зробити щось краще.

Стратегії оброблення помилок та неочікуваних ситуацій

Що може зробити програма за наявності помилок, наприклад, у вхідних даних?
— аварійно завершитись;
— мовчки навернутись (прикинутись, що все гаразд, і завершити виконання без жодних повідомлень);
— зробити щось краще.

Перші два варіанти погані.

Стратегії оброблення помилок та неочікуваних ситуацій

Що може зробити програма за наявності помилок, наприклад, у вхідних даних?
— аварійно завершитись;
— мовчки навернутись (прикинутись, що все гаразд, і завершити виконання без жодних повідомлень);
— зробити щось краще.

Перші два варіанти погані.

За наявності помилок програма
— не повинна аварійно завершуватись;
— повинна якщо не давати точну діагностику проблеми, то принаймні повідомляти, що проблема є.

Стратегії оброблення помилок та неочікуваних ситуацій

Оброблення помилок у функціях

Функція може як перевіряти помилки у вхідних даних, так і не перевіряти.

Обидва підходи можуть мати рацію.

Стратегії оброблення помилок та неочікуваних ситуацій

Оброблення помилок у функціях

Функція може як перевіряти помилки у вхідних даних, так і не перевіряти.
Обидва підходи можуть мати рацію.

У довільному випадку для функції слід задокументувати:
— область визначення (допустимі значення аргументів)
— чи перевіряє вона належність аргументів області визначення
— наслідки застосування до аргументів не з області визначення

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що не виконує перевірок аргументів

За некоректних аргументів:

1) можна отримати сміття або зациклювання коду функції; перед викликом потрібна попередня перевірка

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що не виконує перевірок аргументів

За некоректних аргументів:

1) можна отримати сміття або зациклювання коду функції;
   перед викликом потрібна попередня перевірка
2) можлива поява винятків
   за можливістю бажано задокументувати можливі типи винятків

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що не виконує перевірок аргументів

За некоректних аргументів:
1) можна отримати сміття або зациклювання коду функції; перед викликом потрібна попередня перевірка
2) можлива поява винятків
   за можливістю бажано задокументувати можливі типи винятків

У математичних обчисленнях, коли функція має багаторазово викликатися для значень з деякого проміжку, дешевше за часом один раз перевірити допустимість проміжку і далі виконувати на ньому обчислення функції без додаткових перевірок.

Винятки та перевірки код не пришвидшують.

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що виконує перевірки аргументів

За некоректних аргументів:

1) може повертати заздалегідь фіксоване значення (0, -1, NaN) (має бути задокументовано)

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що виконує перевірки аргументів

За некоректних аргументів:

1) може повертати заздалегідь фіксоване значення (0, -1, NaN) (має бути задокументовано)

2) може генерувати виняток (тип треба задокументувати)

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що виконує перевірки аргументів

За некоректних аргументів:

1) може повертати заздалегідь фіксоване значення (0, -1, NaN) (має бути задокументовано)

2) може генерувати виняток (тип треба задокументувати)

3) може повертати ознаку успішності виконання разом з результатом обчислень

```
bool f(double x, double &y);
```

(треба задокументувати)

Стратегії оброблення помилок та неочікуваних ситуацій

Функція, що виконує перевірки аргументів

За некоректних аргументів:

1) може повертати заздалегідь фіксоване значення (0, -1, NaN) (має бути задокументовано)

2) може генерувати виняток (тип треба задокументувати)

3) може повертати ознаку успішності виконання разом з результатом обчислень

```
bool f(double x, double &y);
```

(треба задокументувати)

4) може повертати код помилки

```
int f(double x, double &y);
```

0 — помилки нема,

не нуль — помилка є

коди помилок мають бути задокументовані